


**PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA**
**MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH**

University of Science and Technology Houari Boumediene (USTHB)
Faculty of Mathematics
Department of Algebra and Number Theory

---

# GRADUATION THESIS

Bachelor in Algebra and Cryptography

## RSA Cryptosystem and Cryptoanalysis

**Presented by:**
Gueddi Mohammed
Saad Aimene
Taha Assassi

**Supervisor:**
Mrs. Oulebsir Assia

**Academic Year: 2025 – 2026**

# Contents


**1 Historical Context and Motivation** **9**

1.1 The Key Distribution Problem . . . . . 9
1.2 Diffie and Hellman's Breakthrough (1976) . . . . . 9
1.3 The Invention of RSA at MIT (1977) . . . . . 10
1.4 The Secret Prior Discovery at GCHQ . . . . . 11
1.5 Why This History Matters for the Rest of This Document . . . . . 11

**2 Mathematical foundations** **13**

2.1 Congruence : . . . . . 13
2.2 Prime Numbers . . . . . 13
2.3 Rings : . . . . . 14
2.4 Ideal . . . . . 14
2.5 Quotient Ring : . . . . . 15
2.6 Units in a Ring . . . . . 16
2.7 The Ring $\mathbb{Z}/n\mathbb{Z}$ . . . . . 16
2.8 GCD and euclide algorithme . . . . . 17
2.9 Bezout's theorem : . . . . . 18
2.10 Extended Euclidean Algorithm : . . . . . 20
2.11 Euler's Totient Function . . . . . 22
2.12 Fermat's Little Theorem . . . . . 23
2.13 Euler's Theorem . . . . . 23
2.14 Chinese Remainder Theorem . . . . . 25
2.15 Primality Tests : . . . . . 27
2.15.1 AKS Primality test : . . . . . 28
2.15.2 Fermat Primality test : . . . . . 32
2.15.3 Miller–Rabin test : . . . . . 34

**3 RSA Cryptosystem** **37**
3.1 RSA Key Generation . . . 37
3.2 Encryption and Decryption : . . . 38
3.3 RSA Proof of Correctness . . . 39
3.4 Fast Modular Exponentiation (square-and-multiply algorithm) . . . 41
3.4.1 Fast Encryption . . . 41
3.4.2 Fast Decryption . . . 44
3.5 Authentication via Digital Signature in RSA . . . 48
3.6 RSA Conceptual Implementation : . . . 48

**4 The Security Foundation of RSA** **53**
4.1 Direct Attack 1 . . . 53
4.2 Direct Attack 2 . . . 54
4.3 Direct Attack 3 : Breaking the Encryption Function . . . 54
4.4 RSA security . . . 55
4.5 Integer Factorization Algorithms . . . 55
4.5.1 Trial Division . . . 56
4.5.2 Fermat's Factorization Method . . . 56
4.5.3 Probabilistic Algorithms: Pollard's Rho and $P-1$ . . . 57
4.5.4 Sub-Exponential Methods . . . 58

**5 RSA cryptanalysis and Padding** **63**
5.1 Wiener's Attack . . . 63
5.1.1 Continued Fractions . . . 63
5.1.2 Applying the Algorithm to RSA . . . 65
5.1.3 How to Avoid Wiener's Attack . . . 69
5.1.4 Generalized Wiener Attacks and the Family of Wiener Attacks . . . 69
5.1.5 Wiener Attack Implementation . . . 70
5.2 Low Public Exponent attacks . . . 71
5.2.1 Hastad's Broadcast Attack . . . 71
5.2.2 Hastad's Stronger Result (Coppersmith's method) . . . 72
5.2.3 Franklin–Reiter Related Message Attack . . . 73
5.2.4 Partial Key Exposure Attack . . . 75
5.3 Homomorphic Attacks . . . 76
5.3.1 Common Modulus Attacks on RSA . . . 77
5.3.2 Chosen cipher text . . . 81
5.3.3 Davida's Attack (Chosen-Message Universal Forgery) . . . 81

5.3.4 LSB Oracle Attack on RSA . . . . . 82
5.4 Padding in RSA . . . . . 85
5.4.1 Why Padding Is Necessary . . . . . 85
5.4.2 What is a Hash Function? . . . . . 86
5.4.3 How Padding Works: Common Principles . . . . . 86
5.4.4 Padding for Encryption: OAEP . . . . . 86
5.4.5 Padding for Signatures . . . . . 87
5.4.6 How Padding Protects Against Core Vulnerabilities . . . . . 89
5.4.7 Conclusion . . . . . 89
5.5 Implementation Attacks . . . . . 89
5.5.1 Timing and Power Attacks on RSA . . . . . 89
5.5.2 Fault-Based Attack on RSA-CRT . . . . . 90
5.5.3 Bleichenbacher's Chosen Ciphertext Attack on RSA . . . . . 92
5.6 Lessons from a Quarter-Century of RSA Attacks . . . . . 95

**6 Conclusion** **97**

# Introduction

The RSA cryptosystem, named after its inventors Ron Rivest, Adi Shamir, and Leonard Adleman, was first published in 1978 and remains, as of 2026, one of the most widely deployed asymmetric encryption schemes.

Unlike symmetric-key cryptography, where two parties must share a secret key before communicating, RSA allows anyone to encrypt a message using a publicly available key. Only the intended recipient who holds the corresponding private key can decrypt it. The security of RSA relies on the presumed difficulty of factoring the product of two large primes a problem for which no efficient classical algorithm is known, The same mathematical concept (modular exponentiation) also enables digital signatures, providing authentication (proof of origin) and non-repudiation (undeniable authorship).

More than four decades after its invention, RSA is still everywhere. It secures TLS certificates that protect web browsing, authenticates SSH connections, signs software updates, and secures DNSSEC. No single cryptosystem has been more impactful and had a longer time in public-key cryptography.

# Chapter 1

# Historical Context and Motivation

## 1.1 The Key Distribution Problem

For most of history, cryptography meant symmetric encryption: two parties sharing the same secret key. This works perfectly once the key is exchanged, but that exchange itself is a vulnerability. Before the 1970s the only options were physical delivery couriers, sealed envelopes, or face-to-face meetings. As computer networks grew, this became a hindrance.

The U.S. National Security Agency maintained a global courier service throughout the Cold War. But as ARPANET [1] connected universities and research centers in the 1970s, physical key distribution became a serious bottleneck. A scalable solution was desperately needed.

## 1.2 Diffie and Hellman's Breakthrough (1976)

In 1976, Whitfield **Diffie** and Martin **Hellman** published "New Directions in Cryptography," a paper that fundamentally re-imagined secure communication. They proposed the concept of ***public-key cryptography***: each user would have two keys, one public and one private. The public key could be published in a directory for anyone to use. The private key would remain secret. this encryption with the public key could only be undone with the private key.

Diffie and Hellman demonstrated a key exchange protocol now called Diffie–Hellman key exchange based on the discrete logarithm problem. This allowed two strangers to agree on

[1] ARPANET was the first operational packet-switching network, created by the U.S. Department of Defense to connect research institutions and enable resource sharing, laying the foundation for the modern Internet

a shared secret over an insecure channel. What ***they did not provide*** was a complete public-key encryption scheme. They explicitly asked whether a trapdoor function based on integer factorization could be constructed. That open question became the challenge that RSA would answer.

## 1.3 The Invention of RSA at MIT (1977)

Ron **R**ivest, Adi **S**hamir, and Leonard **A**dleman at the Massachusetts Institute of Technology spent much of 1977 searching for a practical public-key encryption scheme. Early attempts failed: knapsack-based systems proved insecure, polynomial constructions were too slow. Then **Rivest** had the key insight. Using **Euler's theorem** and **modular exponentiation**, he constructed a scheme where encryption and decryption were both exponentiations modulo a composite number $\boldsymbol{n = p.q}$, the product of two large primes.

The algorithm was elegant. Choose $\boldsymbol{p}$ and $\boldsymbol{q}$ large primes. Compute $\boldsymbol{n = p.q}$ and $\varphi(n) = (p-1)(q-1)$. Select an encryption exponent $e$ coprime to $\varphi(n)$. Compute the decryption exponent $d$ as the modular inverse of $e$ modulo $\varphi(n)$.

$$\boxed{\textbf{Encrypt}} \quad \longrightarrow \quad \boxed{\textbf{Decrypt}}$$

$$\boxed{c = m^e \bmod n.} \qquad \boxed{m = c^d \bmod n.}$$

Security rests on the difficulty of factoring $\boldsymbol{n}$: without $\boldsymbol{p}$ and $\boldsymbol{q}$, an attacker cannot compute $\varphi(n)$ and therefore cannot find $\boldsymbol{d}$ (The Coefficient of Decryption) from $\boldsymbol{e}$ (The Coefficient of Encryption).

Rivest, Shamir, and Adleman published their result in the *Communications of the ACM* in February 1978 . Later that year, Martin Gardner described the algorithm in his "Mathematical Games" column in *Scientific American*. Gardner included a specific challenge ciphertext a 129-digit modulus known as RSA-129 and offered a $100 prize for its factorization. The challenge was not solved until 1994, after a massive distributed effort involving hundreds of volunteers and thousands of computer-years. This dramatic demonstration both proved RSA's resilience and highlighted the rapid advance of factoring algorithms.

## 1.4 The Secret Prior Discovery at GCHQ

In 1997, the British intelligence agency GCHQ declassified documents revealing that public-key cryptography had been invented earlier, in secret. In 1973, a GCHQ mathematician named Clifford Cocks devised a system virtually identical to RSA. His work was classified and never published. When the public RSA appeared in 1978, GCHQ could not acknowledge its own priority without compromising national security.

Historical accounts now recognize Clifford as the first inventor, but the credit for launching public-key cryptography as an open scientific field belongs unequivocally to Rivest, Shamir, and Adleman. The secret British work had no influence on the public development of cryptography, the MIT invention catalyzed the entire discipline.

## 1.5 Why This History Matters for the Rest of This Document

The problems that motivated RSA—secure communication without pre-shared secrets, digital signatures, scalable key management remain central to modern cryptography. The algorithm itself, as originally described in 1978, is mathematically unchanged. But everything around it has evolved: recommended key sizes, padding schemes, resistance to side-channel attacks, and the looming quantum threat.

**The following Chapters 2,3,4 and 5** build on this historical foundation. They explain the mathematics rigorously, describe how RSA is implemented securely today, survey the attacks that have been discovered over forty years. The history tells us ***Why*** RSA exists; the rest of the document tells us ***How*** it works and ***What*** attacks can possibly break it.

# Chapter 2

# Mathematical foundations

## 2.1 Congruence :

**Definition:** If $a$ and $b$ are integers and $m$ is a positive integer, then $a$ is **congruent to** $b$ **modulo** $m$ if $m \mid (a - b)$.

- The notation $a \equiv b \pmod{m}$ says that $a$ is congruent to $b$ modulo $m$.
- We say that $a \equiv b \pmod{m}$ is a **congruence** and that $m$ is its **modulus**.
- Two integers are congruent mod $m$ if and only if they have the same remainder when divided by $m$.
- If $a$ is not congruent to $b$ modulo $m$, we write $a \not\equiv b \pmod{m}$.

**Properties :**

- If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then :

$$a + c \equiv b + d \pmod{m} \quad \text{and} \quad a - c \equiv b - d \pmod{m}.$$

- If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $ac \equiv bd \pmod{m}$

## 2.2 Prime Numbers

A prime number is an integer greater than 1 that has exactly two positive divisors: 1 and itself. **Exemples :** 2, 3, 5, 7, 11.

is 13 a prime number?
We divide it by all prime numbers less than 13. Since it is not divisible by any of these primes 2,3,5,7,11 (wich are all primes lower then 13), then 13 is prime.

## 2.3 Rings :

**Definition :** A ring $A$ is a set with an addition and a multiplication so that:

- $A$ is an abelian group with respect to addition, so $A$ has a zero element denoted 0. Every $x \in A$ has an additive inverse $-x$.
- Multiplication is associative and distributive over addition, i.e.,

$$(xy)z = x(yz), \quad x(y+z) = xy + xz, \quad (x+y)z = xz + yz.$$

**Exemple :** $(\mathbb{Z},+,^*)$ is a ring

## 2.4 Ideal

**Definition :** An ideal of a ring $R$ is a non-empty subset $I$ satisfying :

1. If $x_1, x_2 \in I$, then $x_1 + x_2 \in I$.
2. If $x \in I$ and $r \in R$, then $rx \in I$ and $xr \in I$;

**Exemple :** Let $n \in \mathbb{N}$. Then $(n\mathbb{Z}, +, \cdot)$ is an ideal of $(\mathbb{Z}, +, \cdot)$.

**Proof.** Let $n \in \mathbb{N}$ and consider

$$n\mathbb{Z} = \{nk \mid k \in \mathbb{Z}\}.$$

We show that $n\mathbb{Z}$ is an ideal of $\mathbb{Z}$.

- **Non-empty:** We have $0 = n \cdot 0 \in n\mathbb{Z}$, so $n\mathbb{Z} \neq \emptyset$.
- **Closed under addition:** Let $a, b \in n\mathbb{Z}$. Then there exist $k, \ell \in \mathbb{Z}$ such that $a = nk$ and $b = n\ell$. Hence,

$$a + b = nk + n\ell = n(k + \ell) \in n\mathbb{Z}.$$

- **Absorption by $\mathbb{Z}$:** Let $a \in n\mathbb{Z}$ and $z \in \mathbb{Z}$. Then $a = nk$ for some $k \in \mathbb{Z}$, and

$$za = z(nk) = n(zk) \in n\mathbb{Z}.$$

Thus, $n\mathbb{Z}$ is an ideal of $\mathbb{Z}$.

## 2.5 Quotient Ring :

Let $R$ be a ring and let $I$ be an ideal of $R$. We define a relation on $R$ by

$$a \equiv b \pmod{I} \quad \text{if and only if} \quad a - b \in I.$$

The equivalence class of an element $a \in R$ is given by

$$a + I = \{a + x \mid x \in I\},$$

and the set of all such classes is denoted by

$$R/I.$$

We define addition and multiplication on $R/I$ by

$$(a + I) + (b + I) = (a + b) + I,$$

$$(a + I)(b + I) = ab + I.$$

With these operations, $R/I$ is a ring, called the **quotient ring** of $R$ by $I$ This result is classical in number theory[1]. **Exemple :** $\mathbb{Z}/5\mathbb{Z}$
**Remark :** From Subchapters 1.4 to 1.8, we wanted to introduce $\mathbb{Z}/n\mathbb{Z}$ because we will need it later on proves .

[1]G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers.*

## 2.6 Units in a Ring

Let $R$ be a ring with identity element $1_R$. An element $u \in R$ is called a **unit** if there exists $v \in R$ such that

$$uv = vu = 1_R.$$

The set of all units is denoted by $R^\times$, and it forms a group under multiplication.

This notion comes from standard ring theory[2].

**Example.** In $\mathbb{Z}$, the only units are 1 and $-1$.

**Connection with RSA.** RSA requires computing multiplicative inverses modulo $\varphi(n)$, which means working inside the group of units $(\mathbb{Z}/n\mathbb{Z})^\times$.

## 2.7 The Ring $\mathbb{Z}/n\mathbb{Z}$

Let $n \in \mathbb{N}^*$. The quotient ring $\mathbb{Z}/n\mathbb{Z}$ is defined as the set of equivalence classes:

$$\mathbb{Z}/n\mathbb{Z} = \{0, 1, \ldots, n-1\},$$

where

$$a \equiv b \pmod{n} \iff n \mid (a-b).$$

This construction is standard in number theory and modular arithmetic[3].

**Example.** In $\mathbb{Z}/5\mathbb{Z}$:

$$3 + 4 \equiv 2 \pmod{5}, \quad 2 \cdot 4 \equiv 3 \pmod{5}.$$

**Connection with RSA.** RSA works in $\mathbb{Z}/N\mathbb{Z}$ where $N = pq$. Encryption and decryption are given by:

$$c \equiv m^e \pmod{N}, \qquad m \equiv c^d \pmod{N},$$

with $ed \equiv 1 \pmod{\varphi(N)}$. The existence of $d$ relies on modular inverses computed using the extended Euclidean algorithm.

[2]Basic algebra texts such as D. Perrin, *Cours d'algèbre*.
[3]G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*.

## 2.8 GCD and euclide algorithme

**Definition :** The GCD of two integers is the biggest number that divides both without leaving a remainder. **Euclidean Algorithm**

This is the efficient way to find the GCD. Instead of guessing like it's a lottery, you systematically reduce the problem.

Given two integers $0 < b < a$, we repeatedly apply the division algorithm to obtain a sequence of equations, which eventually terminates with a zero remainder:

$$\begin{aligned} a &= bq_1 + r_1, \quad 0 < r_1 < b, \\ b &= r_1q_2 + r_2, \quad 0 < r_2 < r_1, \\ r_1 &= r_2q_3 + r_3, \quad 0 < r_3 < r_2, \\ &\vdots \\ r_{j-2} &= r_{j-1}q_j + r_j, \quad 0 < r_j < r_{j-1}, \\ r_{j-1} &= r_jq_{j+1}. \end{aligned}$$

The greatest common divisor $\gcd(a, b)$ is $r_j$, the last nonzero remainder in the division process.

**Example :** Find the gcd of 42823 and 6409.

**Solution.**

$$\begin{aligned} 42823 &= 6409 \cdot 6 + 4369 \\ 6409 &= 4369 \cdot 1 + 2040 \\ 4369 &= 2040 \cdot 2 + 289 \\ 2040 &= 289 \cdot 7 + 17 \\ 289 &= 17 \cdot 17 \end{aligned}$$

Therefore,

$$\gcd(42823, 6409) = 17.$$

**Proof of the Euclidean Algorithm.**

Let $a, b \in \mathbb{Z}$ with $b \neq 0$. By the division algorithm, there exist integers $q$ and $r$ such that

$$a = bq + r \quad \text{with} \quad 0 \leq r < |b|.$$

We prove that $\gcd(a, b) = \gcd(b, r)$.

Let $d$ be a common divisor of $a$ and $b$. Then $d \mid a$ and $d \mid b$. Since $r = a - bq$, it follows that

$$d \mid (a - bq) \Rightarrow d \mid r.$$

Thus, every common divisor of $a$ and $b$ is also a common divisor of $b$ and $r$.

Conversely, let $d$ be a common divisor of $b$ and $r$. Then $d \mid b$ and $d \mid r$. Since $a = bq + r$, we get

$$d \mid (bq + r) \Rightarrow d \mid a.$$

Thus, every common divisor of $b$ and $r$ is also a common divisor of $a$ and $b$.

Therefore, the sets of common divisors are the same, and we conclude that.

$$\gcd(a, b) = \gcd(b, r).$$

Since the remainder strictly decreases at each step, the algorithm terminates, and the last nonzero remainder is the greatest common divisor. [4]
**Defenition :** $a$ and $b$ are said to be **coprimer** if: $gcd(a, b) = 1$ and this is what will take us to the Bezout theorem

## 2.9 Bezout's theorem :

Let $a$ and $b$ be integers with at least one of them not zero. Let $d = \gcd(a, b)$ be their greatest common divisor. Then there exist integers $u$ and $v$ such that:

$$au + bv = d$$

[4] Daniel Perrin, *Cours d'algèbre*.

Furthermore, the set of all integers of the form $ax + by$ consists of all the multiples of $d$.

**Remark :**

$$\operatorname{sgn}(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a = 0 \\ -1 & \text{if } a < 0 \end{cases}$$

**proof :**

Consider the set $S$ of all positive linear combinations of $a$ and $b$:

$$S = \{am + bn \mid m, n \in \mathbb{Z}, am + bn > 0\}$$

The set $S$ is non-empty. For example, if $a \neq 0$, then $|a| = a \cdot \operatorname{sgn}(a) + b \cdot 0$ belongs to $S$.

Since $S$ is a non-empty set of positive integers, by the **Well-Ordering Principle**, $S$ contains a smallest element. Let this minimum element be $d$. Since $d \in S$, there exist integers $u$ and $v$ such that:

$$d = au + bv$$

We will now show that $d = \gcd(a, b)$ by proving two conditions:

1. $d$ **is a common divisor of** $a$ **and** $b$**:** By the division algorithm, we can write $a = dq+r$ where $0 \leq r < d$. Substituting for $d$:

   $$r = a - dq = a - (au + bv)q = a(1 - uq) + b(-vq)$$

   This shows that $r$ is a linear combination of $a$ and $b$. If $r > 0$, then $r \in S$. However, $r < d$, and $d$ is defined as the smallest element of $S$. This is a contradiction. Therefore, we must have $r = 0$, which means $d$ divides $a$. By the same logic, $d$ divides $b$.

2. $d$ **is the greatest common divisor:** Let $c$ be any common divisor of $a$ and $b$. Then $c$ divides $a$ and $c$ divides $b$. It follows that $c$ must divide any linear combination of $a$ and $b$. In particular, $c$ divides $au + bv = d$. Since $c$ divides $d$ and $d > 0$, we have $c \leq d$.

Since $d$ is a common divisor and no common divisor is greater than $d$, we conclude that $d = \gcd(a, b)$.

[5]

**Remark :** To find the u and v from the definition, we should use an algorithm named the Extended Euclidean Algorithm.

[5] Daniel Perrin, *Cours d'algèbre*.

## 2.10 Extended Euclidean Algorithm :

In this note we are given two positive integers $a > b$. Recall that Euclid's algorithm finds $gcd(a, b)$ by repeated division, with the greatest common divisor being the last nonzero remainder. The Euclidean algorithm is quite easy to follow. The extended Euclidean algorithm uses data found during the Euclidean algorithm to find solutions $x$ and $y$ to the equation :

$$ax + by = gcd(a, b).$$

We define sequences $(r_i)$, $(x_i)$, and $(y_i)$ as follows:

$$r_0 = a, \quad r_1 = b$$

$$x_0 = 1, \quad x_1 = 0$$

$$y_0 = 0, \quad y_1 = 1$$

For $i \geq 1$, perform the division:

$$r_{i-1} = q_i r_i + r_{i+1}$$

and update:

$$x_{i+1} = x_{i-1} - q_i x_i$$

$$y_{i+1} = y_{i-1} - q_i y_i$$

The algorithm stops when $r_{n+1} = 0$. Then:

$$\gcd(a, b) = r_n$$

and the integers $x_n$ and $y_n$ satisfy:

$$ax_n + by_n = \gcd(a, b).$$

**Exemple :**

$$ax + by = gcd(a, b).$$

with $a = 2011$ and $b = 1534$ , define $x$ and $y$ ?

we first should use the Euclidean Algorithm :

$$\begin{aligned} 2011 &= 1534 \cdot 1 + 477 \\ 1534 &= 477 \cdot 3 + 103 \\ 477 &= 103 \cdot 4 + 65 \\ 103 &= 65 \cdot 1 + 38 \\ 65 &= 38 \cdot 1 + 27 \\ 38 &= 27 \cdot 1 + 11 \\ 27 &= 11 \cdot 2 + 5 \\ 11 &= 5 \cdot 2 + 1 \\ 5 &= 1 \cdot 5 \end{aligned}$$

Notice how the remainders migrate to the left from one line to the next. Now we turn to solving the equation

$$ax + by = 1.$$

First, the basic idea: We are trying to write the greatest common divisor (which is the last remainder, 1 in this case) as a (linear) combination of $a$ and $b$. Clearly, the first line in the algorithm expresses the first remainder as such a combination:

$$477 = a - b.$$

And the next line expresses the next remainder 103 as a combination of the previous two remainders 1534 and 477, and thus as a combination of $a$ and $b$,

$$103 = b - 477 \cdot 3 = b - (a - b) \cdot 3 = -a + 4b$$

and so we can proceed downwards until we reach the bottom.

However, we can effectively halve the work by reducing everything modulo one of the original numbers! Let $\equiv$ stand for congruence modulo $a = 2011$. On the left below is the original Euclid's algorithm, on the right is the new calculation. Notice that the new name of the game is to write each remainder as a multiple of $b$ modulo $a$. Each line on the right is first a rewrite on the line on its left, then simplifying using data from the two lines above (except for the opening moves):

$$
\begin{aligned}
2011 &= 1534 \cdot 1 + 477 & 477 &\equiv -1534 = -b \\
1534 &= 477 \cdot 3 + 103 & 103 &= 1534 - 477 \cdot 3 \equiv b - (-b) \cdot 3 = 4b \\
477 &= 103 \cdot 4 + 65 & 65 &= 477 - 103 \cdot 4 \equiv -b - 4b \cdot 4 = -17b \\
103 &= 65 \cdot 1 + 38 & 38 &= 103 - 65 \cdot 1 \equiv 4b - (-17b) = 21b \\
65 &= 38 \cdot 1 + 27 & 27 &= 65 - 38 \cdot 1 \equiv -17b - 21b = -38b \\
38 &= 27 \cdot 1 + 11 & 11 &= 38 - 27 \cdot 1 \equiv 21b - (-38b) = 59b \\
27 &= 11 \cdot 2 + 5 & 5 &= 27 - 11 \cdot 2 \equiv -38b - 59b \cdot 2 = -156b \\
11 &= 5 \cdot 2 + 1 & 1 &= 11 - 5 \cdot 2 \equiv 59b - (-156b) \cdot 2 = 371b
\end{aligned}
$$

we have in fact shown that

$$371b \equiv 1 \pmod{a},$$

in other words, that 371 is the inverse of 1534 modulo 2011. This sort of result is of course one of the reasons to perform the extended Euclidean algorithm in the first place, so if that were our goal, we could stop here.

But we wanted to find $x$ and $y$. From what we have shown, we are pretty certain that $371b - 1$ is a multiple of $a$. So we perform the division:

$$\frac{371b - 1}{a} = \frac{371 \cdot 1534 - 1}{2011} = 283$$

with no remainder, which of course confirms our calculation and can also be rearranged as

$$283a - 371b = 1,$$

so a solution to the problem is

$$x = 283, \quad y = -371.$$

[A practical guide to the extended Euclid algorithm][6]

## 2.11 Euler's Totient Function

Euler's totient function $\varphi(n)$ counts the number of positive integers less than $n$ that are coprime with $n$, i.e., integers $k$ such that $\gcd(k, n) = 1$.

[6] Hanche-Olsen, *Extended Euclid Algorithm.*

**Properties:**

- If $p$ is prime, $\varphi(p) = p - 1$.
- If $p$ is prime and $k \geq 1$, $\varphi(p^k) = p^k - p^{k-1}$.
- If $m$ and $n$ are coprime, $\varphi(mn) = \varphi(m) \cdot \varphi(n)$.

**Example:** $\varphi(9) = 6$, since 1, 2, 4, 5, 7, 8 are coprime with 9.

**Applications:** Euler's totient function is used in number theory, modular arithmetic, and cryptography, especially in RSA encryption..

## 2.12 Fermat's Little Theorem

**Theorem:** If $p$ is a prime number and $a$ is not divisible by $p$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

**Corollary:** If $p - 1$ divides $m$, then

$$a^m \equiv 1 \pmod{p},$$

**Applications:** primality testing, cryptography and cryptoanalysis.

**Example 1:** Compute $2^6 \mod 7$. Here $p = 7$ (prime) and $\gcd(2, 7) = 1$. By Fermat's theorem:

$$2^{7-1} = 2^6 \equiv 1 \pmod{7}.$$

So $2^6 \mod 7 = 1$. Furthermore $2^{6k} \mod 7 = 1$ for all $k \in \mathbb{N}$ .

## 2.13 Euler's Theorem

**Theorem:** If $a$ is coprime with $n$ (i.e., $\gcd(a, n) = 1$), then

$$a^{\varphi(n)} \equiv 1 \pmod{n},$$

where $\varphi(n)$ is Euler's totient function (counts integers $\leq n$ coprime with $n$).

**Applications:** cryptography, RSA, modular arithmetic simplifications.

*Proof.* [7] The proof rests on a single key observation: multiplication by $a$ permutes the reduced residue system modulo $n$.
Let $\{r_1, r_2, \ldots, r_{\varphi(n)}\}$ be the reduced residue system modulo $n$. Consider the set of products

$$S = \{ar_1 \bmod n,\ ar_2 \bmod n,\ \ldots,\ ar_{\varphi(n)} \bmod n\}.$$

We verify two properties:

- *Coprimality.* Since $\gcd(a, n) = 1$ and $\gcd(r_j, n) = 1$, we have $\gcd(ar_j, n) = 1$ for each $j$.
- *Distinctness.* If $ar_i \equiv ar_j n$, then since $\gcd(a, n) = 1$ we may cancel $a$ to get $r_i \equiv r_j n$, and since $r_i, r_j \in \{1, \ldots, n\}$, this forces $r_i = r_j$.

We have $\varphi(n)$ distinct nonzero residues, all coprime to $n$, so $S = \{r_1, \ldots, r_{\varphi(n)}\}$ as sets modulo $n$.

Since $S$ is a permutation of the reduced residue system, their products are equal modulo $n$:

$$\prod_{j=1}^{\varphi(n)} (ar_j) \ \equiv \ \prod_{j=1}^{\varphi(n)} r_j \mod n.$$

Factoring out the $\varphi(n)$ copies of $a$ on the left:

$$a^{\varphi(n)} \prod_{j=1}^{\varphi(n)} r_j \ \equiv \ \prod_{j=1}^{\varphi(n)} r_j \mod n.$$

Let $P = \prod_{j=1}^{\varphi(n)} r_j$. Since every $r_j$ satisfies $\gcd(r_j, n) = 1$, their product satisfies $\gcd(P, n) = 1$, so $P$ is invertible modulo $n$. Canceling $P$ from both sides:

$$a^{\varphi(n)} \equiv 1 \mod n$$

□

[7] Perrin, *Cours d'algèbre*.

[8]

**Exemple :** Compute $2^{100} \mod 35$ using Fermat's and Euler's theorems?

We have $35 = 5 \cdot 7$, so it is composite. $\gcd(2, 35) = 1$, so Euler's theorem applies.

Compute $\varphi(35)$:

$$\varphi(35) = \varphi(5) \cdot \varphi(7) = (5-1)(7-1) = 4 \cdot 6 = 24$$

By Euler's theorem:

$$2^{24} \equiv 1 \pmod{35}.$$

Now write $100 = 24 \cdot 4 + 4$, so:

$$2^{100} = (2^{24})^4 \cdot 2^4 \equiv 1^4 \cdot 2^4 \equiv 16 \pmod{35}.$$

$$2^{100} \equiv 16 \pmod{35}$$

**Remark:** Here, we use Euler's theorem for the composite modulus. If the modulus is prime, Fermat's theorem would be enough.

## 2.14 Chinese Remainder Theorem

**Theorem (Chinese Remainder Theorem) :** Suppose that modulo $n_1, \ldots, n_k$ are pairwise coprime. Then the simultaneous congruences

$$x \equiv b_1 \pmod{n_1}, \quad x \equiv b_2 \pmod{n_2}, \quad \ldots, \quad x \equiv b_k \pmod{n_k}$$

have a unique solution modulo $N := n_1 \cdots n_k$.

Specifically, for each $i$, define $N_i = \frac{N}{n_i}$ and compute its inverse $\lambda_i$ such that

$$\lambda_i N_i \equiv 1 \pmod{n_i},$$

[8] Daniel Perrin, *Cours d'algèbre*.

then

$$x \equiv b_1\lambda_1 N_1 + b_2\lambda_2 N_2 + \cdots + b_k\lambda_k N_k \pmod{N}.$$

*Proof.* **Existence.** Set $N = \prod_{i=1}^{k} n_i$ and, for each $i$, define

$$N_i = \frac{N}{n_i} = \prod_{j \neq i} n_j.$$

Since the $n_j$ are pairwise coprime and each $N_i$ is a product of all moduli *except* $n_i$, every prime factor of $n_i$ is absent from $N_i$. Hence $\gcd(N_i,\, n_i) = 1$.

By Bézout's identity, there exists an integer $M_i$ satisfying

$$N_i M_i \equiv 1 \pmod{n_i}.$$

We call $M_i$ the *modular inverse* of $N_i$ modulo $n_i$. Now define the candidate solution:

$$x = \sum_{i=1}^{k} a_i N_i M_i.$$

Fix an index $i$. For each $j \neq i$, the factor $n_i$ appears explicitly in $N_j$, so $n_i \mid N_j$, and therefore $N_j M_j \equiv 0 n_i$. Every term in the sum with $j \neq i$ thus vanishes modulo $n_i$, leaving:

$$x \;\equiv\; a_i N_i M_i \;\equiv\; a_i \cdot 1 \;=\; a_i n_i.$$

Since $i$ was arbitrary, $x$ is a simultaneous solution to the entire system.

**Uniqueness.** Suppose $x$ and $y$ are both solutions. Then $n_i \mid (x - y)$ for every $i$. Since the $n_i$ are pairwise coprime, their product divides $x - y$ as well,[9] giving $N \mid (x - y)$, hence $x \equiv yN$. □

**Exemple :** We solve the system:

$$x \equiv 5 \pmod{50}, \quad x \equiv 17 \pmod{31}.$$

[9]This step uses the following standard lemma: if $m_1, \ldots, m_k$ are pairwise coprime and each $m_i \mid d$, then $m_1 m_2 \cdots m_k \mid d$. A proof by induction on $k$ is given in Ireland and Rosen [**?**]

The moduli 50 and 31 are pairwise coprime, so the theorem applies. We compute:

$$N = 50 \cdot 31 = 1550, \quad N_1 = 31, \quad N_2 = 50.$$

We must therefore solve:

$$\begin{cases} 31\lambda_1 \equiv 1 \pmod{50} \Rightarrow \lambda_1 \equiv 21 \\ 50\lambda_2 \equiv 1 \pmod{31} \Rightarrow \lambda_2 \equiv 18 \end{cases}$$

Finally,

$$x \equiv b_1\lambda_1 N_1 + b_2\lambda_2 N_2$$

$$x \equiv 5 \cdot 21 \cdot 31 + 17 \cdot 18 \cdot 50$$

$$x \equiv 1505 \pmod{1550}.$$

**Remark.** The Chinese Remainder Theorem is useful in simplifying computations in RSA calculations.

## 2.15 Primality Tests :

### Definition

A primality test is an algorithm used to determine whether a given integer $n > 1$ is a prime number or a composite number. We differentiate two types of primality tests :

- **Deterministic primality tests**
  - Always give a correct answer (prime or composite)
  - No probability of error
  - Based on exact mathematical proofs
  - Example: AKS primality test
- **Probabilistic primality tests**
  - Give correct answer with high probability
  - May have a very small error probability

- Much faster for large numbers
- Error probability decreases with repetition
- Examples: Miller–Rabin test, Fermat test

### 2.15.1 AKS Primality test :

The AKS primality test is an algorithm used to determine whether a number is prime. It was introduced in 2002 by Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. The main characteristic of the AKS algorithm is that it is deterministic, meaning it always gives a correct answer without relying on probability. It is also the first primality test proven to run in polynomial time (it can solve the problem in a number of steps that grows at a manageable rate as the input size increases). Despite its theoretical importance, the AKS algorithm is not commonly used in practice because it is slower than other methods. However, it remains an important result in computer science and number theory. the algorithme is based on the next theorem :

**remark** If $x \in \mathbb{R}$, we denote by $\lfloor x \rfloor$ the greatest integer $k$ such that $k \leq x$ and similarly $\lceil x \rceil$ the smallest integer $k$ such that $k \geq x$.

From this definition we have:

$$x - 1 < \lfloor x \rfloor \leq x \quad \text{and} \quad x \leq \lceil x \rceil < x + 1.$$

**Theorem :**Let $n \in \mathbb{N}$, $n \geq 2$, and $a \in \mathbb{Z}$ an integer coprime with $n$. Then $n$ is prime if and only if:

In $\mathbb{Z}_n[X]$,

$$(X + a)^n = X^n + a$$

**Proof.** We have:

$$(X + a)^n = X^n + a^n + \sum_{k=0}^{n} C_k^n X^k a^{n-k} \Rightarrow (X + a)^n - (X^n + a^n) = \sum_{k=0}^{n} C_k^n X^k a^{n-k}$$

$$(\Rightarrow)$$

For all $k \in \mathbb{N}$ such that $0 < k < n$, the coefficient of $(X + a)^n - (X^n + a^n)$ is $C_k^n a^{n-k}$.

For all $k \in [|1, n-1|]$,

$$kC_k^n = nC_{k-1}^{n-1} \Rightarrow n \text{ divides } kC_k^n$$

and since $\gcd(k, n) = 1$,

$$n \text{ divides } C_k^n$$

Thus,

$$(X+a)^n - (X^n + a^n) \equiv 0\ [n],$$

but since $a$ is coprime with $n$, by Fermat's theorem we obtain:

$$a^n = a\ [n]$$

Hence,

$$(X+a)^n - (X^n + a) \equiv 0\ [n]$$

$$(\Leftarrow)$$

Conversely

Suppose that we have:

$$(X+a)^n \equiv (X^n + a)[n],$$

and let $d$ be a proper divisor of $n$ such that

$$1 \leq d < n.$$

We have:

$$C_d^n = \frac{n}{d} C_{d-1}^{n-1} \equiv 0\ [n]$$

**Residues of $C_i^{n-1}$ modulo $n$:**

For $i = 1$, $C_1^{n-1} = n - 1 \equiv -1[n]$

From Pascal's formula, we have:

$$C_i^{n-1} + C_{i+1}^{n-1} = C_{i+1}^n \equiv 0[n]$$

$$\Rightarrow C_{i+1}^{n-1} \equiv -C_i^{n-1}[n]$$

$$\Rightarrow C_i^{n-1} \equiv (-1)^i [n], \quad \forall 1 \leq i \leq n-1$$

Thus,

$$C_d^n = \frac{n}{d} C_{d-1}^{n-1} = \frac{n}{d}(-1)^{d-1} \equiv 0[n]$$

The last equality is true only if $d = 1$.

The method proposed above is a simple test that checks whether an odd number is prime or not. It is based on a generalization of Fermat's theorem to polynomial rings over finite fields. Although this method is quite simple, its complexity is $\Omega(n)$ when evaluating the $n$ coefficients in the worst case, which is not polynomial time. A simple way to reduce this complexity is to evaluate both sides of the equation modulo a polynomial of the form $X^r - 1$, for a suitable choice of $r$, and thus the complexity will depend on the size of $r$, that is

$$(X + a)^n \equiv (X^n + a) \mod (X^r - 1, n) \qquad (*)$$

In other words, it consists of computing:

$$(X + a)^n \mod X^r - 1 \quad \text{and} \quad X^n + a \mod (X^r - 1) = X^{n \bmod r} + a$$

For every prime integer $n$, equality $(*)$ is verified for any values of $r$ and $a$. The converse is problematic in the sense that for some composite numbers $n$, equality $(*)$ may still hold for certain values of $r$ and $a$. [10]

**Remark.** The notation $(X^r - 1, n)$ refers to computations in the quotient ring $(\mathbb{Z}/n\mathbb{Z})[X]/(X^r - 1)$, where coefficients are reduced modulo $n$ and polynomials are reduced modulo $X^r - 1$.

[10] [1] Ngoupeyou Tondji Lionel, *Mémoire de Master II (Algèbre)*, Université de Yaoundé I, 2017–2018.

**Algorithm 1** AKS Primality Test

**Input:** $n \in \mathbb{N},\ n \geq 2$

**Output:** "prime" or "composite"

**if** $n = a^b$ *for some* $a \in \mathbb{N}$ *and* $b > 1$ **then**
  **return** *composite*

Find the smallest $r$ such that $\mathrm{ord}_r(n) > \log^2(n)$

**for** $a \leftarrow 2$ **to** $r$ **do**
  **if** $1 < \gcd(a, n) < n$ **then**
    **return** *composite*

**if** $n \leq r$ **then**
  **return** *prime*

**for** $a \leftarrow 1$ **to** $\left\lfloor \sqrt{\varphi(r)} \log(n) \right\rfloor$ **do**
  **if** $(X + a)^n \not\equiv X^n + a \pmod{(X^r - 1,\ n)}$ **then**
    **return** *composite*

**return** *prime*

**Remark.** The notation $\mathrm{ord}_r(n)$ denotes the multiplicative order of $n$ modulo $r$, that is, the smallest positive integer $k$ such that

$$n^k \equiv 1 \pmod{r}.$$

**Example :** Let us take as an example a large prime number that we already know is prime:

$$n = 1000003.$$

We apply the AKS primality test to verify its primality.

**1. Perfect power test.** We check whether $n = a^b$ for some $a \in \mathbb{N}$ and $b > 1$. The number 1000003 is not a perfect power, so we proceed.

**2. Choice of $r$.** We choose the smallest integer $r$ such that

$$\mathrm{ord}_r(n) > \log^2(n).$$

Since $\log^2(1000003) \approx 400$, a suitable $r$ is selected as guaranteed by the algorithm.

**3. GCD test.** We check $\gcd(a, n)$ for integers $a \leq r$. Since 1000003 is prime, we have $\gcd(a, 1000003) = 1$ for all $a < n$.

**4. Size check.** Since $n > r$, we continue.

**5. Polynomial test.** We verify whether

$$(X + a)^n \equiv X^n + a \pmod{(X^r - 1, n)}$$

for all required values of $a$. Since $n$ is prime, the binomial structure ensures that the identity holds.

**Conclusion.** All conditions are satisfied, hence

1000003 is prime.

### 2.15.2 Fermat Primality test :

he Fermat primality test is one of the simplest methods used to check whether a number is prime. It is based on Fermat's Little Theorem, introduced by Pierre de Fermat , the Little Theorem of Fermat is the mathematical backbone of this ptimality test , For a number $n > 1$, the test checks whether:

$$a^{n-1} \equiv 1 \pmod{n}$$

for some number $a$ that is coprime with $n$. If this is not true, then $n$ is definitely composite. If it is true, then $n$ is probably prime.

The test is called probabilistic because it can make mistakes. Sometimes a composite number can still satisfy the formula and "pretend" to be prime. The error comes from the fact that some values of $a$ give the correct result even when $n$ is not prime.

In general, for a composite number, at least half of the choices of $a$ will detect that it is not prime. This means that if we repeat the test several times with different values of $a$, the probability of error becomes very small:

$$\text{Error} \leq \left(\frac{1}{2}\right)^k$$

after $k$ independent tests.

So, the more times you test, the safer the result becomes.

**Algorithm 2** Fermat Primality Test

**Input:** $n > 1$

**Output:** "composite" or "probably prime"

Choose randomly an integer $a \in \{2, 3, \ldots, n-2\}$

Compute $a^{n-1} \bmod n$

**if** $a^{n-1} \bmod n \neq 1$ **then**

  **return** *composite*

**return** *probably prime*

**Example.** Let $n = 127$. We apply the Fermat primality test.

We choose $a = 2 \in \{2, 3, \ldots, 125\}$. We need to compute:

$$2^{126} \bmod 127.$$

We compute step by step using successive squaring:

$$2^2 = 4$$

$$2^4 = 16$$

$$2^8 = 16^2 = 256 \equiv 2 \pmod{127}$$

$$2^{16} = (2^8)^2 \equiv 2^2 = 4 \pmod{127}$$

$$2^{32} = (2^{16})^2 \equiv 4^2 = 16 \pmod{127}$$

$$2^{64} = (2^{32})^2 \equiv 16^2 = 256 \equiv 2 \pmod{127}$$

Now we decompose:

$$126 = 64 + 32 + 16 + 8 + 4 + 2$$

So:

$$2^{126} = 2^{64} \cdot 2^{32} \cdot 2^{16} \cdot 2^8 \cdot 2^4 \cdot 2^2$$

Substitute the values modulo 127:

$$2^{126} \equiv 2 \cdot 16 \cdot 4 \cdot 2 \cdot 16 \cdot 4 \pmod{127}$$

Now multiply step by step:

$$2 \cdot 16 = 32$$

$$32 \cdot 4 = 128 \equiv 1 \pmod{127}$$

$$1 \cdot 2 = 2$$

$$2 \cdot 16 = 32$$

$$32 \cdot 4 = 128 \equiv 1 \pmod{127}$$

Thus:

$$2^{126} \equiv 1 \pmod{127}$$

Since the condition

$$a^{n-1} \equiv 1 \pmod{n}$$

is satisfied, the test declares that 127 is *probably prime*.

Repeating the test with other values of $a$ gives the same result. Therefore:

$$127 \text{ is prime.}$$

### 2.15.3 Miller–Rabin test :

the algorithm builds on earlier ideas in number theory and computational complexity. It was developed in the 1970s ,it's a probabilistic primality test that is based on the Little Theorem of Fermat as it's foundation and this formula :

$$n - 1 = 2^s \cdot d, \quad \text{where } d \text{ is odd and n is the integer we want to test}$$

, the strength of the algorithm comes from the fact that composite numbers tend to reveal themselves under repeated random tests, making false positives extremely unlikely when enough iterations are used. Because of this balance between speed and reliability, Miller–Rabin is widely used in cryptography, especially in generating large prime numbers for encryption systems like RSA.

**Algorithm 3** Miller–Rabin Primality Test

An integer $n > 2$ "composite" or "probably prime"

**if** $n$ *is even* **then**
  **return** composite
Factor $n - 1$ as $2^s \cdot t$ with $t$ odd
$s \leftarrow 0 \;\; t \leftarrow n - 1$
**while** $t$ *is even* **do**
  $s \leftarrow s + 1 \;\; t \leftarrow t/2$
Choose $x \in \{2, \ldots, n-2\}$ uniformly at random
Compute $x^t \bmod n$
**if** $x^t \not\equiv 1 \pmod{n}$ **then**
  **if** $x^t \not\equiv -1 \pmod{n}$ **then**
    **for** $i = 1$ **to** $s$ **do**
      $x \leftarrow x^2 \bmod n$
      **if** $x \equiv 1 \pmod{n}$ **then**
        **return** composite
      **if** $x \equiv -1 \pmod{n}$ **then**
        **break**
    **return** composite
**return** probably prime

**Error in the Miller–Rabin Test :** After $k$ independent iterations, the Miller–Rabin primality test has a worst-case probability of error bounded by $(1/4)^k$, making it highly reliable in practice. **Example of the Miller–Rabin Test for** $n = 82\,589\,933$

We apply Algorithm 1 to the integer $n = 82\,589\,933$.

First, since $n$ is odd, we continue. We factor

$$n - 1 = 82\,589\,932 = 2^2 \cdot 20\,647\,483,$$

so that

$$s = 2 \quad \text{and} \quad t = 20\,647\,483.$$

Choose $x = 2$. Then

$$x^t \bmod n = 2^{20\,647\,483} \bmod 82\,589\,933 = 71\,093\,132.$$

[10] [1] Introduction to Algorithms (CS 482), Cornell University, Instructor: Bobby Kleinberg, Lecture Notes, 5 May 2010, *The Miller–Rabin Randomized Primality Test*.

Since

$$71\,093\,132 \not\equiv 1 \pmod{n} \quad \text{and} \quad 71\,093\,132 \not\equiv -1 \pmod{n},$$

we continue with the squaring step:

$$x \leftarrow x^2 \bmod n = 71\,093\,132^2 \bmod 82\,589\,933 = 82\,589\,932.$$

Now

$$82\,589\,932 \equiv -1 \pmod{n}.$$

Therefore, the algorithm returns **probably prime** and in fact it is prime.

# Chapter 3

# RSA Cryptosystem

## 3.1 RSA Key Generation

RSA key generation is based on selecting two large prime numbers, $p$ and $q$, which are used to compute the modulus $n = p \cdot q$. These primes are not chosen manually or deterministically, but generated using a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG). The randomness of $p$ and $q$ is essential because RSA security depends on their unpredictability. If an attacker can predict or reproduce these primes, they can factor $n$ and recover the private key.

**Randomness** in cryptography is evaluated through entropy, which measures uncertainty in a source. High entropy means the output is highly unpredictable, while low entropy indicates structure or bias. In practice, randomness is assessed using statistical test suites such as NIST SP 800-22, Dieharder, or TestU01. These tools check properties like uniform distribution, independence of bits, and absence of detectable patterns. Modern systems generate randomness using operating system CSPRNGs such as `/dev/urandom` on Linux or `BCryptGenRandom` on Windows. These generators collect entropy from physical and system-level noise, including timing variations, hardware events, and environmental fluctuations. This entropy is continuously mixed and expanded into secure random output used for cryptographic purposes. Once random candidates are generated, they are tested for primality using probabilistic algorithms such as Miller–Rabin, which efficiently confirm whether a number behaves like a prime with extremely high confidence.The size of RSA keys is also critical to security. The modulus $n$ is typically 2048 bits or larger in modern systems. This means each prime $p$ and $q$ is about 1024 bits long. Higher security levels use 3072-bit or 4096-bit keys, where primes are correspondingly larger. Smaller sizes like 1024-bit RSA are

considered insecure because advances in computation and factoring methods make them vulnerable. Larger key sizes increase the difficulty of factoring $n$ exponentially, making attacks computationally infeasible with current technology. However, key size alone is not sufficient: even large keys are insecure if randomness is weak or predictable.

After computing $n = p \cdot q$ and $\varphi(n) = (p-1)(q-1)$, we select a public exponent $e$ such that $\gcd(e, \varphi(n)) = 1$. This pair $(e, n)$ becomes the public key. In practice, $e = 65537$ is the near-universal choice. Why? It is the Fermat prime $2^{16}+1$, which has only two '1' bits in its binary representation ($10000000000000001_2$), making encryption (modular exponentiation) very fast. At the same time, it is large enough to thwart small-exponent attacks that historically affected tiny values like $e = 3$. Using this well-studied constant balances efficiency and security, which is why modern standards recommend it.

The private key $d$ is then computed as the modular inverse of $e$ modulo $\varphi(n)$,

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

This is efficiently found using the extended Euclidean algorithm. Unlike the public key, $d$ must be kept strictly confidential. It is typically stored in an encrypted keystore or a hardware security module (HSM), with access restricted to authorized processes or personnel. Compromising $d$ allows an attacker to decrypt any message intended for the key owner, effectively breaking the entire cryptosystem. Hence, securing the storage and handling of the private key is as vital as generating strong primes in the first place.

## 3.2 Encryption and Decryption :

**RSA Encryption**

Given the public key $(e, n)$ and the plain-text $m$, the encryption function $E$ is:

$$C = E(m) \equiv m^e \pmod{n}$$

where $C$ is the cipher text.

**Remark :** the sender encrypts the message he wants to send using the receiver's public key.

**RSA Decryption**

Given the private key $d$ and the ciphertext $C$, the decryption function is:

$$D(C) \equiv y^d \equiv m \pmod{n}$$

| Receiver | Sender |
|---|---|
| **Key Creation** | |
| Choose secret primes $p$ and $q$.<br>Choose encryption exponent $e$<br>with $\gcd(e,(p-1)(q-1))=1$.<br>Compute $d$ satisfying<br>$ed \equiv 1 \pmod{(p-1)(q-1)}$.<br>Publish $N=pq$ and $e$. | |
| **Encryption** | |
| | Choose plaintext $m$.<br>Use public key $(N,e)$<br>to compute $c \equiv m^e \pmod{N}$.<br>Send ciphertext $c$. |
| **Decryption** | |
| Compute $m' \equiv c^d \pmod{N}$.<br>Then $m'=m$. | |

RSA cryptosystem in brief

So in summary the RSA cryptosystem depends on the following dichotomy:

- **Setup.** Let $p$ and $q$ be large primes, let $N=pq$, and let $e$ and $c$ be integers.
- **Problem.** Solve the congruence $x^e \equiv c \pmod{N}$ for the variable $x$.
- **Easy.** The receiver, who knows the values of $p$ and $q$, can easily solve for $x$.
- **Hard.** An adversary, who does not know the values of $p$ and $q$, cannot easily find $x$.
- **Dichotomy.** Solving $x^e \equiv c \pmod{N}$ is easy for a only the party who possesses the decomposition of $n$, but it is hard for all other parties.

## 3.3 RSA Proof of Correctness

Recall the RSA setup: one chooses two distinct large primes $p$ and $q$, computes the modulus $n=pq$ and Euler's totient $\varphi(n)=(p-1)(q-1)$, selects a public exponent $e$ with $\gcd(e,\varphi(n))=1$, and computes the private exponent $d \equiv e^{-1} \pmod{\varphi(n)}$. A plaintext

$m \in \{0, 1, \ldots, n-1\}$ is encrypted as $C \equiv m^e \pmod{n}$, and decryption recovers $m$ by computing $C^d \pmod{n}$.

The key identity to verify is therefore

$$C^d \equiv (m^e)^d = m^{ed} \equiv m \pmod{n}.$$

Because $d$ was chosen so that

$$ed \equiv 1 \pmod{\varphi(n)}$$

there exists an integer $k$ such that

$$ed = 1 + k\varphi(n)$$

The proof then splits naturally into two cases depending on whether $\gcd(m, n) = 1$ or not.

**Case 1:** $\gcd(m, n) = 1$

This is the generic case and occurs with overwhelming probability, since the proportion of integers in $\{0, 1, 2, \ldots, n-1\}$ that share a factor with $n$ is negligibly small when $p$ and $q$ are large.

Since $\gcd(m, n) = 1$, then by Euler's theorem

$$m^{\varphi(n)} \equiv 1 \pmod{n}$$

therefore

$$m^{ed} \equiv m^{1+k\varphi(n)} = m \cdot 1^k \equiv m \pmod{n}$$

**Case 2:** $\gcd(m, n) \neq 1$

Although this case breaks the RSA security unintentionally (it reveals the decomposition of $n$ by simply calculating $\gcd(C, n)$ because $\gcd(m, n) \neq 1 \implies \gcd(m, n) = p \implies \exists k \in \mathbb{N}$ such that $m = k \cdot p$ this means that $C \equiv m^e \equiv (k \cdot p)^e \pmod{n} \implies p \mid C \implies \gcd(C, n) = p$) and the fact that this case is extremely rare in practice, a complete proof of RSA correctness must address it. Since $n = pq$ with $p$ and $q$ prime, and since $0 \leq m < n$, the only non-trivial possibilities are $p \mid m$ or $q \mid m$, but not both (as that would require $n \mid m$, contradicting $m < n$).
By symmetry, assume without loss of generality that $p \mid m$ and $\gcd(m, q) = 1$, so $m \equiv 0 \pmod{p}$, and therefore $m \equiv m^{ed} \equiv 0 \pmod{p}$.

Since $\gcd(m, q) = 1$, Fermat's little theorem gives $m^{q-1} \equiv 1 \pmod{q}$. Now write $ed = 1 + k(p-1)(q-1)$. Since $(q-1)$ divides $(p-1)(q-1)$, we have

$$m^{ed} = m \cdot \left(m^{q-1}\right)^{k(p-1)} \equiv m \cdot 1 = m \pmod{q}.$$

We have simultaneously

$$m^{ed} \equiv m \pmod{p} \quad \text{and} \quad m^{ed} \equiv m \pmod{q}.$$

Since $p$ and $q$ are distinct primes, $\gcd(p, q) = 1$, so by the Chinese Remainder Theorem there exists a unique residue modulo $n = pq$ satisfying both congruences. Since $m$ itself satisfies both, we conclude $m^{ed} \equiv m \pmod{n}$.

## 3.4 Fast Modular Exponentiation (square-and-multiply algorithm)

Public-key cryptosystems such as RSA rely on modular exponentiation with very large numbers, which makes efficient computation essential for practical use. A direct method of exponentiation is far too slow because it would require an enormous number of multiplications when the exponent is large, as is the case in RSA, where the public exponent e and especially the private exponent d can be hundreds or thousands of bits long. To solve this problem, RSA uses the square-and-multiply algorithm, which computes exponentiation by scanning the binary representation of the exponent from left to right. At each step, the current result is squared, and if the current bit is 1, it is also multiplied by the base. This method greatly reduces the number of operations, since the number of squarings depends only on the bit-length of the exponent, and the number of multiplications depends on the number of 1 bits in its binary form. On average, for an exponent of bit-length t+1, the algorithm requires t squarings and about 0.5t multiplications. Compared with straightforward exponentiation, which is computationally impossible for cryptographic exponents, square-and-multiply makes RSA encryption and decryption feasible on modern computers.

### 3.4.1 Fast Encryption

RSA encryption requires computing $c \equiv m^e \pmod{n}$, where $e$ is a large integer. Naive repeated multiplication would require $e - 1$ multiplications, which is completely infeasible for practical key sizes. The standard solution is the *square-and-multiply* algorithm, also

known as fast modular exponentiation or binary exponentiation.

### Algorithm Description

The key observation is that computing $m^e \pmod{n}$ can be decomposed by reading the binary representation of the exponent $e = \sum_{i=0}^{t} e_i 2^i$, where $e_i \in \{0, 1\}$ and $e_t = 1$ is the most significant bit. The algorithm processes the bits of $e$ from left to right (most significant to least significant), maintaining a running result $R$ and performing at most one multiplication per bit.[1]

**Algorithm 4** Square-and-Multiply (Left-to-Right Binary Exponentiation)

**Input:** Base $m$, exponent $e > 0$, modulus $n$; binary representation $e = (e_t e_{t-1} \dots e_0)_2$

**Output:** $R \equiv m^e \pmod{n}$

$R \leftarrow 1$ **for** $i \leftarrow t$ *downto* $0$ **do**

  $R \leftarrow R^2 \bmod n$ `/* Square step */`

  **if** $e_i = 1$ **then**

    $R \leftarrow R \cdot m \bmod n$ `/* Multiply step */`

  **else**

    `/* Do nothing */`

**return** $R$

### Proof of Correctness

**Claim.** Algorithm 4 returns $m^e \bmod n$ after at most $2\lfloor \log_2 e \rfloor$ modular multiplications.

*Proof.* Let $R_i$ denote the value of $R$ after processing bit $e_i$. We show by induction that, after processing bits $e_t, e_{t-1}, \dots, e_i$, the accumulated exponent of $m$ in $R$ equals the integer $(e_t e_{t-1} \cdots e_i)_2$.

*Base case.* Before any iteration, $R = 1 = m^0$, corresponding to an empty binary prefix.

*Inductive step.* Suppose before processing bit $e_i$ the running exponent is $v = (e_t \cdots e_{i+1})_2$. After the squaring step, the exponent becomes $2v$, and if $e_i = 1$ the subsequent multiply step adds 1, yielding $2v + e_i = (e_t \cdots e_i)_2$. This matches the binary prefix extended by one bit, completing the induction.

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1997, p. 461. Knuth describes this technique as: *"one of the most important algorithms in all of computational mathematics"*.

After processing all $t+1$ bits, the accumulated exponent is $(e_t \cdots e_0)_2 = e$, so $R \equiv m^e \pmod{n}$.

Regarding complexity: the loop executes exactly $t+1 = \lfloor \log_2 e \rfloor + 1$ iterations, each involving one squaring and at most one multiplication, giving at most $2\lfloor \log_2 e \rfloor + 2$ modular multiplications in total.[2] □

**Worked Example**

Compute $c \equiv 7^{13} \pmod{33}$.

The binary representation of 13 is $(1101)_2$. We proceed as follows:

| Step | Bit $e_i$ | Square: $R \leftarrow R^2 \bmod 33$ | Multiply: $R \leftarrow R \cdot 7 \bmod 33$ | $R$ |
|---|---|---|---|---|
| Init | – | – | – | 1 |
| $i=3$ | 1 | $1^2 = 1$ | $1 \cdot 7 = 7$ | 7 |
| $i=2$ | 1 | $7^2 = 49 \equiv 16$ | $16 \cdot 7 = 112 \equiv 13$ | 13 |
| $i=1$ | 0 | $13^2 = 169 \equiv 4$ | (skipped) | 4 |
| $i=0$ | 1 | $4^2 = 16$ | $16 \cdot 7 = 112 \equiv 13$ | 13 |

Therefore $7^{13} \equiv 13 \pmod{33}$, achieved in 6 modular operations instead of 12.

[2] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, *op. cit.*, p. 615. The authors note: *"this method requires at most* $2(\lfloor \log_2 e \rfloor)$ *modular multiplications, compared to* $e-1$ *for naïve exponentiation"*.

**Python Implementation**

```python
def fast_encrypt(m: int, e: int, n: int) -> int:
    """
    Compute m^e mod n using the square-and-multiply method.
    Equivalent to Python's built-in pow(m, e, n).
    """
    result = 1
    m = m % n
    while e > 0:
        if e % 2 == 1:            # current bit is 1 -> multiply
            result = (result * m) % n
        e >>= 1                   # shift to next bit
        m = (m * m) % n           # square the base
    return result

# Example: encrypt m=7 with public key (e=13, n=33)
ciphertext = fast_encrypt(7, 13, 33)
print(f"Ciphertext: {ciphertext}")   # Output: 13
```

### 3.4.2 Fast Decryption

RSA decryption computes $m \equiv c^d \pmod{n}$, where $d$ is typically as large as the modulus $n$ itself (2048 bits or more). While the square-and-multiply algorithm already reduces this to $O(\log d)$ multiplications modulo $n$, each such multiplication involves numbers of size $\sim n$. A considerably faster approach – reducing execution time by a factor of approximately four in practice – exploits the known factorization $n = pq$ via the Chinese Remainder Theorem. [3]

#### Algorithm Description

The idea is to compute $c^d$ separately modulo $p$ and modulo $q$, working with integers roughly half the size of $n$, and then recombine the two results via the CRT.
Define the reduced private exponents

$$d_p \equiv d \pmod{p-1}, \qquad d_q \equiv d \pmod{q-1},$$

and the CRT coefficient $q_{\text{inv}} \equiv q^{-1} \pmod{p}$.

[3] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed., CRC Press, 2020, p. 427. The authors remark: *"CRT-based decryption gives a roughly four-fold speedup over direct computation of* $c^d$ mod $n$*"*.

**Algorithm 5** Fast RSA Decryption via CRT (Garner's Method)

**Require:** Ciphertext $c$, primes $p$, $q$, exponents $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, coefficient $q_{\text{inv}} = q^{-1} \bmod p$

**Ensure:** Plaintext $m \equiv c^d \pmod{n}$

0: $m_p \leftarrow c^{d_p} \bmod p$ {Half-size exponentiation mod $p$}

0: $m_q \leftarrow c^{d_q} \bmod q$ {Half-size exponentiation mod $q$}

0: $h \leftarrow q_{\text{inv}} \cdot (m_p - m_q) \bmod p$ {Garner recombination}

0: $m \leftarrow m_q + h \cdot q$

0: **return** $m$ =0

### Proof of Correctness

**Claim.** Algorithm 5 returns $m \equiv c^d \pmod{n}$.

*Proof.* We first verify the partial results. By Fermat's little theorem, for any $c$ with $\gcd(c, p) = 1$,

$$c^{p-1} \equiv 1 \pmod{p},$$

so $c^d = c^{d_p + k(p-1)} \equiv c^{d_p} \pmod{p}$, giving $m_p \equiv c^d \pmod{p}$. An identical argument yields $m_q \equiv c^d \pmod{q}$.

We must then find $m$ such that $m \equiv m_p \pmod{p}$ and $m \equiv m_q \pmod{q}$. The Garner recombination computes[4]

$$h \equiv q_{\text{inv}}(m_p - m_q) \pmod{p},$$

and sets $m = m_q + hq$. We verify:

- **Modulo $q$:** $m = m_q + hq \equiv m_q \pmod{q}$.
- **Modulo $p$:** $m \equiv m_q + hq \equiv m_q + q_{\text{inv}}(m_p - m_q) \cdot q \pmod{p}$. Since $q_{\text{inv}} \cdot q \equiv 1 \pmod{p}$, this simplifies to $m \equiv m_q + (m_p - m_q) = m_p \pmod{p}$.

By the CRT, since $\gcd(p, q) = 1$, the solution is unique modulo $n = pq$. Because $m_q < q$ and $0 \le h < p$, we have $0 \le m = m_q + hq < q + pq = n + q$, and since $m < n$ by construction, no final reduction is needed.

[4]R. Garner, "The residue number system," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 2, pp. 140–147, 1959. Garner's original formulation states: *"the mixed-radix representation provides an efficient method for CRT reconstruction without large intermediate values"*.

Since $m_p \equiv c^d \pmod{p}$ and $m_q \equiv c^d \pmod{q}$, and $c^d \bmod n$ is also the unique solution modulo $n$, we conclude $m = c^d \bmod n$. □

[Speedup Analysis] Each of the two exponentiations in Algorithm 5 operates on a modulus of $\sim \log_2 p \approx k/2$ bits (where $k = \log_2 n$) with an exponent of similar size. The cost of a modular multiplication with a $b$-bit modulus scales as $O(b^2)$ (or $O(b \log b \log\log b)$ with fast multiplication). Replacing one $k$-bit exponentiation by two $(k/2)$-bit ones reduces the multiplication cost by approximately $2 \times (1/2)^2 = 1/2$, giving a theoretical speedup of factor 4 over direct computation.[5]

**Worked Example**

Let $p = 11$, $q = 13$, so $n = 143$, $\varphi(n) = 120$. Choose $e = 7$; then $d \equiv 7^{-1} \pmod{120} = 103$. Suppose $c = 6$ is the ciphertext.

**Precomputed values:**

$$d_p = 103 \bmod 10 = 3, \quad d_q = 103 \bmod 12 = 7, \quad q_{\text{inv}} = 13^{-1} \bmod 11 \equiv 6 \pmod{11}.$$

**Step 1 – Partial exponentiations:**

$$m_p = 6^3 \bmod 11 = 216 \bmod 11 = 7, \quad m_q = 6^7 \bmod 13.$$

Computing $6^7 \bmod 13$ by squaring: $6^2 = 36 \equiv 10$, $6^4 \equiv 100 \equiv 9$, $6^7 = 6^4 \cdot 6^2 \cdot 6^1 \equiv 9 \cdot 10 \cdot 6 = 540 \equiv 8 \pmod{13}$.

**Step 2 – Garner recombination:**

$$h = 6 \cdot (7 - 8) \bmod 11 = 6 \cdot (-1) \bmod 11 = -6 \bmod 11 = 5.$$

$$m = 8 + 5 \cdot 13 = 8 + 65 = 73.$$

**Verification:** $c^d = 6^{103} \bmod 143$. Using Python's `pow(6,103,143)` gives 73. ✓

[5] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, *op. cit.*, p. 612.

**Python Implementation**

```python
def mod_inverse(a: int, m: int) -> int:
    """Extended Euclidean algorithm to compute a^{-1} mod m."""
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist.")
    return x % m

def extended_gcd(a: int, b: int):
    if a == 0:
        return b, 0, 1
    g, x, y = extended_gcd(b % a, a)
    return g, y - (b // a) * x, x

def fast_decrypt_crt(c: int, p: int, q: int, d: int) -> int:
    """
    RSA decryption using the Chinese Remainder Theorem (Garner's method).
    Approximately 4x faster than direct pow(c, d, p*q).
    """
    n   = p * q
    d_p = d % (p - 1)                  # reduced exponent mod p
    d_q = d % (q - 1)                  # reduced exponent mod q
    q_inv = mod_inverse(q, p)          # CRT coefficient

    m_p = pow(c, d_p, p)               # half-size exponentiation mod p
    m_q = pow(c, d_q, q)               # half-size exponentiation mod q

    h = (q_inv * (m_p - m_q)) % p     # Garner recombination
    m = m_q + h * q                    # final plaintext

    return m % n

# ------------------------------------------------------------------
# Example: p=11, q=13, e=7, d=103, ciphertext c=6
p, q, d, c = 11, 13, 103, 6
plaintext = fast_decrypt_crt(c, p, q, d)
print(f"Decrypted: {plaintext}")       # Output: 73

# Verify against direct decryption
n = p * q
assert plaintext == pow(c, d, n), "CRT decryption mismatch!"
print("Verification passed.")
```

**Remark**

This example shows how RSA encryption and decryption rely on repeated modular squaring and selective multiplication. Even though the exponents are large in real systems, the square-and-multiply algorithm reduces the complexity from linear to logarithmic in the number of bits of the exponent, making RSA practically usable.

## 3.5 Authentication via Digital Signature in RSA

RSA digital signatures provide a way to verify the authenticity and integrity of a message. The process relies on a pair of keys: a **private key**, known only to the signer, and a **public key**, available to anyone. To sign a message $m$, the sender computes a signature using the private key $(d, n)$ as follows:

$$\sigma = m^d \mod n$$

The signature $\sigma$ is then sent along with the message. To verify the signature, the receiver uses the sender's public key $(e, n)$ and computes:

$$m' = \sigma^e \mod n$$

If $m'$ matches the original message $m$, the signature is considered valid. This works because of the mathematical relationship between the keys in RSA: encrypting with one key and decrypting with the other always returns the original value. Since only the sender knows the private key, a valid signature proves that the message was signed by them and has not been altered in transit.

## 3.6 RSA Conceptual Implementation :

**Remark:** The implementation above corresponds to what is known as textbook RSA — a purely mathematical description intended for instructional purposes. In real-world systems, using textbook RSA directly is unsafe. In the following chapters, we will explore various attack that exploit even minor implementation flaws, demonstrating why secure RSA requires careful attention to padding, key generation, and side-channel resistance.

Listing 3.1: Basic RSA Implementation in Python

```
1 import random
```

```
2
3  # ------------------ Miller-Rabin Primality Test ------------------
4  def is_prime(n, k=5):
5      """Return True if n is probably prime (error probability 2^{-k})."""
6      if n < 2:
7          return False
8      if n in (2, 3):
9          return True
10     if n % 2 == 0:
11         return False
12
13     # Write n-1 = d * 2^s
14     d = n - 1
15     s = 0
16     while d % 2 == 0:
17         d //= 2
18         s += 1
19
20     # Test k witnesses
21     for _ in range(k):
22         a = random.randrange(2, n - 1)
23         x = pow(a, d, n)
24         if x == 1 or x == n - 1:
25             continue
26         for _ in range(s - 1):
27             x = (x * x) % n
28             if x == n - 1:
29                 break
30         else:
31             return False
32     return True
33
34 def generate_prime(bits=8):
35     """Generate a random prime of roughly 'bits' bits."""
36     while True:
37         p = random.getrandbits(bits)
38         # ensure it is odd and large enough
39         p |= (1 << bits - 1) | 1
40         if is_prime(p):
41             return p
42
43 # ------------------ Extended Euclidean Algorithm ------------------
44 def egcd(a, b):
```

```
45      """Return (g, x, y) such that a*x + b*y = g = gcd(a, b)."""
46      if a == 0:
47          return (b, 0, 1)
48      else:
49          g, x1, y1 = egcd(b % a, a)
50          return (g, y1 - (b // a) * x1, x1)
51
52  def modinv(a, m):
53      """Return the modular inverse of a modulo m (assumes gcd(a,m)=1)."""
54      g, x, _ = egcd(a, m)
55      if g != 1:
56          raise ValueError("Inverse does not exist")
57      return x % m
58
59  # ------------------  S q u a r e andMultiply  ( L e f t toRight )
        ------------------
60  def square_and_multiply(base, exp, mod):
61      """Return base^exp mod mod using binary exponentiation."""
62      result = 1
63      bits = bin(exp)[2:]   # e.g., '1011'
64      for bit in bits:
65          result = (result * result) % mod      # square
66          if bit == '1':
67              result = (result * base) % mod    # multiply
68      return result
69
70  # ------------------ Garner's CRT for RSA Decryption ------------------
71  def rsa_decrypt_crt(c, p, q, d):
72      """
73      Fast RSA decryption using Garner's method.
74      Assumes n = p*q, ed     1 mod   (n).
75      """
76      d_p = d % (p - 1)
77      d_q = d % (q - 1)
78      q_inv = modinv(q, p)
79
80      m_p = square_and_multiply(c, d_p, p)   # c^d_p mod p
81      m_q = square_and_multiply(c, d_q, q)   # c^d_q mod q
82
83      # Garner recombination
84      h = (q_inv * (m_p - m_q)) % p
85      m = m_q + h * q
86      return m
```

```
87
88  # ------------------ RSA Key Generation ------------------
89  def generate_rsa_keys(bits=8):
90      """Generate RSA keys. For demo only     uses small primes."""
91      p = generate_prime(bits)
92      q = generate_prime(bits)
93      while q == p:
94          q = generate_prime(bits)
95      n = p * q
96      phi = (p - 1) * (q - 1)
97
98      # Choose e (usually 65537, here small for demo)
99      e = 65537
100     if phi % e == 0:
101         e = 257
102     d = modinv(e, phi)
103
104     public_key = (e, n)
105     private_key = (d, p, q)
106     return public_key, private_key
107
108 # ------------------ Encryption ------------------
109 def encrypt(msg, public_key):
110     """Encrypt a message (integer) using RSA public key (e, n)."""
111     e, n = public_key
112     return square_and_multiply(msg, e, n)
113
114 # ------------------ Decryption (with CRT) ------------------
115 def decrypt(cipher, private_key):
116     """Decrypt ciphertext using  C R T based  fast decryption."""
117     d, p, q = private_key
118     return rsa_decrypt_crt(cipher, p, q, d)
119
120 # ------------------ Demonstration ------------------
121 if __name__ == "__main__":
122     print("=== Basic RSA Implementation (Textbook RSA) ===\n")
123
124     public_key, private_key = generate_rsa_keys(bits=8)
125     e, n = public_key
126     print(f"Public key:  (e = {e}, n = {n})")
127     print(f"Private key: (d = {private_key[0]} kept secret, p = {
        private_key[1]}, q = {private_key[2]})")
128
```

```
129     original = 42
130     print(f"\nOriginal message: {original}")
131 
132     cipher = encrypt(original, public_key)
133     print(f"Encrypted ciphertext: {cipher}")
134 
135     decrypted = decrypt(cipher, private_key)
136     print(f"Decrypted plaintext:  {decrypted}")
137 
138     assert original == decrypted, "Decryption failed!"
139     print("\ n   Encryption / decryption successful.")
```

# Chapter 4

# The Security Foundation of RSA

The security of the RSA cryptosystem rests on a single, well-studied assumption: the Integer Factorization Problem (IFP) is computationally infeasible for sufficiently large composite numbers when those numbers are generated correctly. Given a modulus $n = p \cdot q$, where $p$ and $q$ are large, distinct primes of roughly equal size, no known classical procedure can recover $p$ and $q$ within a practical time frame. For properly implemented RSA – using key lengths of 2048 bits or more, strong random prime generation, and secure padding schemes – this assumed hardness guarantees that an adversary possessing only the public key $(n, e)$ cannot derive the private decryption exponent $d$. However, the claim that RSA is secure requires more than an appeal to the difficulty of factoring; it demands an examination of every plausible path an attacker might take to break the system. The original analysis of RSA identified three direct cryptanalytic approaches, each of which attempts to avoid factoring $n$ explicitly. A detailed examination shows that all three are, in fact, computationally equivalent to factoring $n$. Consequently, the security of RSA stands or falls with the intractability of the IFP.

## 4.1 Direct Attack 1

One obvious way to break RSA is to compute Euler's totient function $\varphi(n) = (p-1)(q-1)$ directly from the public modulus $n$, without first finding $p$ and $q$. If an attacker could obtain $\varphi(n)$, then the private exponent $d$ would be immediately recoverable as the modular inverse of the public exponent $e$ modulo $\varphi(n)$, using the extended Euclidean algorithm. At first glance, computing $\varphi(n)$ might seem easier than factoring $n$ because $\varphi(n)$ is a function of the unknown primes. However, the two problems are computationally equivalent. Given $\varphi(n)$

and $n$, an attacker can easily factor $n$ as follows:

- Compute $p + q = n - \varphi(n) + 1$.
- Compute $p - q = \sqrt{(p+q)^2 - 4n}$.
- Then $p = \frac{(p+q)+(p-q)}{2}$ and $q = \frac{(p+q)-(p-q)}{2}$.

Thus, any method that yields $\varphi(n)$ can be transformed directly into a factoring algorithm. Since factoring is widely believed to be hard, computing $\varphi(n)$ without factoring is equally hard. This equivalence also explains why $n$ must be composite; if $n$ were prime, $\varphi(n) = n-1$ would be trivial to compute, and RSA would offer no security.

## 4.2 Direct Attack 2

A second direct attack attempts to find the decryption exponent $d$ itself, bypassing both factoring $n$ and computing $\varphi(n)$. The attacker knows that $d$ satisfies $e \cdot d \equiv 1 \pmod{\varphi(n)}$, but $\varphi(n)$ is unknown. One might imagine searching for $d$ directly, perhaps by brute force or by solving some congruence. However, if an attacker discovers any integer $d$ that works as a valid decryption exponent – even one different from the original $d$ chosen by the legitimate user – then the system is broken. Finding any such $d$ is no easier than factoring $n$ and the reason is that knowledge of $d$ allows the attacker to compute $e \cdot d - 1$, which is a multiple of $\varphi(n)$. Once a multiple of $\varphi(n)$ is known, a classic result (due to Miller) shows that $n$ can be factored efficiently using probabilistic methods. Therefore, the problem of determining $d$ reduces directly to the problem of factoring $n$. Moreover, all valid decryption exponents differ by multiples of $\text{lcm}(p-1, q-1)$; finding any one of them yields the same factoring capability. Consequently, a brute-force search for $d$ is hopeless for large key sizes, and any shortcut that finds $d$ would imply an efficient factoring algorithm – something not known to exist.

## 4.3 Direct Attack 3 : Breaking the Encryption Function

A third and more general cryptanalytic approach does not target $\varphi(n)$ or $d$ explicitly. Instead, the attacker attempts to decrypt a given ciphertext $c$ directly by computing the $e$-th root of $c$ modulo $n$, i.e., finding $m$ such that $m^e \equiv c \pmod{n}$. This is precisely the problem of inverting the RSA encryption function without knowledge of the private key. Unlike fac-

toring, the problem of computing $e$-th roots modulo a composite number is not a classical, well-studied hard problem with centuries of history. Nevertheless, the original RSA analysis argues that this problem is likely to be as difficult as factoring, though a formal proof has never been established. The conjecture is that any general method that can compute $e$-th roots modulo $n$ without knowing the factorization could be converted into an efficient factoring algorithm. If such a conversion were proven, then breaking RSA would be provably equivalent to factoring. However, the equivalence remains unproven; it is possible (though considered unlikely by most cryptographers) that computing $e$-th roots is easier than factoring, even if no such method is known today. This gap between the root-extraction problem and the factoring problem is an active area of research. For practical purposes, after decades of failed attempts to find an efficient root-extraction method that bypasses factoring, the overwhelming consensus is that the two problems are equally hard for properly chosen RSA parameters.

## 4.4 RSA security

The security of RSA ultimately depends on the practical difficulty of the Integer Factorization Problem (IFP), to which the three direct attacks described above all reduce. Over the centuries, mathematicians have developed a wide variety of factorization algorithms—from trial division and Fermat's difference of squares to methods exploiting smoothness (e.g., Pollard's p1), random walks (Pollard's ), elliptic curves, and eventually the General Number Field Sieve (GNFS). Despite this rich and evolving toolkit, every classical factoring algorithm suffers from super-polynomial or sub-exponential time complexity. For a 2048-bit RSA modulus (about 617 decimal digits), the GNFS—the best known classical method—requires an astronomically large number of operations, far beyond the reach of current or foreseeable classical computers. The historical record of factoring progress confirms only slow, incremental gains; each additional bit of key size demands exponentially more effort to break. Consequently, when RSA is implemented correctly with sufficiently large keys, the IFP remains a reliable one-way function against classical adversaries.

## 4.5 Integer Factorization Algorithms

None of the algorithms in this section can break a properly sized RSA key—say, 2048 bits or more. Their running times are simply too slow. But that does not make them unimportant. These algorithms break small moduli quite easily, and small moduli still appear in practice: old systems, weak key generation, or coding mistakes can all lead to keys that are just a

few hundred bits long. For such cases, methods like Pollard's  or the quadratic sieve work surprisingly well. Understanding them also helps explain why larger keys are safe and how the more powerful General Number Field Sieve operates. So, while these algorithms are no threat to modern RSA, they are worth studying—because they break what is breakable, and they teach us why the rest is secure.

### 4.5.1 Trial Division

Test divisibility of $n$ by every prime number up to $\sqrt{n}$. If a divisor is found, it is a prime factor; the process continues on the quotient.

**Algorithm (conceptual):** 1. Let $p = 2$.
2. While $p \times p \leq n$: - If $p$ divides $n$, output $p$ as a factor and replace $n$ by $n/p$. - Else, move $p$ to the next prime.
3. If the remaining $n > 1$, output $n$ as a prime factor.

For an RSA modulus $n$ of 200 decimal digits, $\sqrt{n}$ has about 100 digits. Even testing only primes (which are much fewer than all integers) would require an astronomical number of divisions – far beyond any conceivable computational resource.

### 4.5.2 Fermat's Factorization Method

If a composite number $n$ can be expressed as a difference of two squares, then the factors are directly obtained. Fermat observed that any odd composite $n = p \cdot q$ can be written as:

$$n = \left(\frac{p+q}{2}\right)^2 - \left(\frac{p-q}{2}\right)^2 = a^2 - b^2 = (a-b)(a+b)$$

The algorithm searches for integers $a$ and $b$ satisfying this equation.

**Algorithm (conceptual):**
1. Set $a = \lceil \sqrt{n} \rceil$.
2. Compute $b^2 = a^2 - n$.
3. If $b^2$ is a perfect square, then $b = \sqrt{b^2}$ and the factors are $a - b$ and $a + b$.
4. Otherwise, increment $a$ by 1 and repeat from step 2.

Fermat's method is very fast when the two factors $p$ and $q$ are close to each other (so that $a$ is only slightly larger than $\sqrt{n}$). However, in properly implemented RSA, $p$ and $q$ are chosen to be far apart (typically differing by several orders of magnitude), making Fermat's method

as slow as trial division.

### 4.5.3 Probabilistic Algorithms: Pollard's Rho and $P-1$

In the 1970's, John Pollard introduced two algorithms that use randomness and number-theoretic properties to find factors much faster than classical methods, especially when the factor $p$ is relatively small or has a special form.

#### Pollard's Rho Algorithm

The algorithm defines a pseudo-random function $f(x) = (x^2 + c) \mod n$ (with a small constant $c$, often 1). Starting from an initial value $x_0$ (say 2), the function generates a sequence: $x_1 = f(x_0)$, $x_2 = f(x_1)$, and so on. Because the sequence is computed modulo $n$, but we are interested in the behavior modulo a prime factor $p$ of $n$, the sequence eventually repeats after about $\sqrt{p}$ steps due to the birthday paradox. This repetition creates a cycle. When two values in the sequence are congruent modulo $p$ but not modulo $n$, their difference is a multiple of $p$. Computing the greatest common divisor (gcd) of that difference with $n$ reveals the factor $p$.

**Algorithm (conceptual):**
1. Choose a random starting value $x$ and a constant $c$ (e.g., $c = 1$).
2. Set $y = x$ (a second pointer that moves more slowly).
3. Repeat:
- Update $x = (x^2 + c) \mod n$ (fast pointer).
- Update $y = (y^2 + c) \mod n$ once, then again (slow pointer, so after two updates it moves one step).
- Compute $g = \gcd(|x - y|, n)$.
- If $1 < g < n$, then $g$ is a non-trivial factor of $n$. Return $g$.
- If $g = n$, restart with different $c$ or starting values.
4. Continue until a factor is found or a time limit is reached.

The expected number of iterations is proportional to $\sqrt{p}$, where $p$ is the smallest prime factor of $n$. Thus, if $p$ is, say, 10 digits long, the algorithm finds it in about $10^5$ steps – very fast. However, if both factors are large (e.g., 100 digits each), $\sqrt{p}$ is still $10^{50}$, which is impossible.

#### Pollard's $p-1$ Algorithm

This algorithm exploits Fermat's little theorem: for a prime $p$ and any integer $a$ not divisible by $p$, we have $a^{p-1} \equiv 1 \pmod{p}$. If we can find a number $M$ that is a multiple of $p-1$, then

$a^M \equiv 1 \pmod{p}$, which means that $p$ divides $a^M - 1$. Therefore, $\gcd(a^M - 1, n)$ will be at least $p$. The trick is to choose $M$ as the least common multiple of all integers up to some bound $B$ (written $M = \text{lcm}(1, 2, 3, \ldots, B)$). If all prime factors of $p - 1$ are $\leq B$ (i.e., $p - 1$ is "$B$-smooth"), then $M$ is a multiple of $p - 1$, and the gcd will reveal $p$.

**Algorithm (conceptual):**
1. Choose a bound $B$ (e.g., $10^6$).
2. Compute $M = \text{lcm}(1, 2, 3, \ldots, B)$ (or simply multiply all primes up to $B$ raised to appropriate powers).
3. Choose a random base $a$ (e.g., 2).
4. Compute $x = a^M \mod n$ using modular exponentiation.
5. Compute $g = \gcd(x - 1, n)$.
6. If $1 < g < n$, then $g$ is a non-trivial factor.
If $g = 1$, increase $B$ or choose a different $a$.
If $g = n$, try a smaller $B$ or a different $a$.

The algorithm is extremely fast when $p - 1$ is smooth – that is, composed only of small prime factors. For example, many primes used in older implementations had this property. However, modern RSA key generation avoids such primes by ensuring that both $p - 1$ and $q - 1$ have at least one large prime factor. The $p - 1$ algorithm then becomes ineffective because the required bound $B$ would need to be as large as the largest prime factor of $p - 1$, making the computation of $M$ and $a^M$ infeasible.

### 4.5.4 Sub-Exponential Methods

The algorithms in this section represent the state of the art for classical factorization before the advent of quantum computing. They are based on a common strategy called the **congruence of squares** and achieve sub-exponential running times – meaning they are faster than any exponential algorithm but still much slower than polynomial time.

#### The Congruence-of-Squares Method in Integer Factorization and Smooth Numbers

Its core idea, first proposed as a general scheme by Kraitchik, is to find two integers $x$ and $y$ such that

$$x^2 \equiv y^2 \pmod{n} \quad \text{and} \quad x \not\equiv \pm y \pmod{n}.$$

When such a pair is found, $\gcd(x - y, n)$ yields a non-trivial factor of $n$. The challenge lies in efficiently producing $x$ and $y$ without exhaustive search.

The natural method is to generate many random integers $z_i$ and compute their squares modulo $n$, hoping that the product of some subset forms a perfect square. In practice, one works with the values $z_i^2 \bmod n$ and tries to factor them over a fixed set of small primes (the **factor base**). A number that factors completely over the factor base is called **smooth**. The idea of using smooth numbers for factorization was advanced by Morrison and Brillhart, who showed that collecting enough smooth congruences allows one to combine them via linear algebra (over $\mathbb{F}_2$) to obtain a congruence of squares.

Smooth numbers play a critical role because they are easy to work with: their prime factorisation is known, and their exponents can be manipulated to produce a square. The efficiency of modern algorithms such as Dixon's random-squares algorithm and the Quadratic Sieve depends on how quickly smooth numbers are found. Dixon's algorithm simply picks random numbers and tests them for smoothness; the Quadratic Sieve dramatically speeds up this search by using a sieving process to identify smooth values without testing each candidate individually.

This combination – Kraitchik's congruence principle, Morrison and Brillhart's smooth-number accumulation, and sieving techniques – underpins nearly all sub-exponential factoring algorithms, including the Quadratic Sieve and the Number Field Sieve.

### Quadratic Sieve (QS)

**Core idea:** The Quadratic Sieve seeks two integers $x$ and $y$ such that $x^2 \equiv y^2 \pmod{n}$ but $x \not\equiv \pm y \pmod{n}$. Then $\gcd(x - y, n)$ gives a non-trivial factor of $n$. To find such a pair, the algorithm generates many numbers of the form $Q(t) = (t + \lfloor \sqrt{n} \rfloor)^2 - n$ for various integers $t$. These values are relatively small (on the order of $\sqrt{n}$). The algorithm then tries to factor each $Q(t)$ completely using a fixed set of small primes called the **factor base**. A number that factors completely over the factor base is called **smooth**. Once enough smooth numbers are collected, linear algebra (Gaussian elimination) over $GF(2)$ is used to combine them so that the product of the corresponding $Q(t)$ values becomes a perfect square. That product gives the desired congruence $x^2 \equiv y^2 \pmod{n}$.

The Quadratic Sieve was the fastest classical algorithm for factoring numbers up to about 100 decimal digits. Its main weakness is that the size of the factor base and the number of smooth values required grow with the size of $n$, making it impractical for very large numbers (hundreds of digits).

### Elliptic Curve Method (ECM)

**Core idea:** ECM generalizes Pollard's $p-1$ algorithm by replacing the multiplicative group modulo $p$ with the group of points on a random elliptic curve over the finite field $\mathbb{F}_p$. In Pollard's $p-1$, the success condition depends on the smoothness of $p-1$, which is fixed for a given prime. In ECM, the group order of an elliptic curve modulo $p$ is not $p-1$ but some integer in the interval $(p-2\sqrt{p}+1, p+2\sqrt{p}+1)$ (by Hasse's theorem). By choosing many different random curves, the algorithm effectively samples many different group orders. Eventually, with high probability, it finds a curve whose order is smooth (i.e., composed only of small primes). When that happens, the same exponentiation process as in $p-1$ (but using elliptic curve point multiplication) will reveal the factor $p$.

The running time of ECM depends primarily on the size of the smallest prime factor $p$ of $n$, not on the size of $n$ itself. This makes ECM ideal for finding medium-sized factors (up to about 70 decimal digits) in large numbers. It has been used, for example, to find factors of Fermat numbers and to clean up "hard" composites before applying the General Number Field Sieve.

### General Number Field Sieve (GNFS)

The GNFS is the most powerful classical algorithm for factoring large integers (over 100 decimal digits). It extends the congruence-of-squares idea of the Quadratic Sieve but works in a much more abstract setting: algebraic number fields. Instead of working directly with integers modulo $n$, the algorithm constructs two polynomial equations with a common root modulo $n$. Then it searches for pairs of numbers that are smooth in two different number fields simultaneously. When enough such pairs are found, a combination yields a congruence of squares modulo $n$.

The algorithm requires working with rings of integers, ideals, and logarithms in number fields, but despite this complexity, the GNFS has been used to set all modern factoring records, including the factorization of RSA-768 (a 232-digit number) in 2009 and RSA-250 (250 digits) in 2020.

**Limitations for RSA security:** The GNFS still requires sub-exponential time, and for a 2048-bit RSA modulus (about 617 decimal digits), the estimated number of operations remains astronomically high – far beyond the reach of any classical computer that could be built with foreseeable technology. Therefore, as long as the modulus is large enough and generated without exploitable structure, classical factorization algorithms do not break RSA.

Summary of integer factorization algorithms

| **Algorithm** | **Year** | **Complexity type** | **Max key size (bits / decimal digits)** |
|---|---|---|---|
| Trial division | / | Expo | $\approx$ 60 bits (18 digits) |
| Fermat's Method | 1643 | Expo | $\approx$ 80 bits (24 digits) |
| Pollard's $p-1$ | 1974 | Expo | $\approx$ 100 bits (30 digits) if $p-1$ smooth |
| Pollard's $\rho$ | 1975 | Expo | $\approx$ 120 bits (36 digits) for smallest factor |
| ECM | 1987 | Sub-expo | $\approx$ 280 bits (85 digits) for smallest factor |
| QS | 1984 | Sub-expo | $\approx$ 330 bits (100 decimal digits) |
| GNFS | 1993 | Sub-expo | $\approx$ 829 bits (250 decimal digits) – current record |
| Shor's algorithm | 1994 | Polynomial | Any key size (with fault-tolerant quantum computer) |

# Chapter 5

# RSA cryptanalysis and Padding

## 5.1 Wiener's Attack

In order to reduce decryption time (or signature-generation time), one may wish to use a small value of d rather than a random d. Since modular exponentiation takes time linear in $\log_2 d$, a small d can improve performance by at least a factor of 10 (for a 1024 bit modulus). Unfortunately, a clever attack due to M. Wiener shows that a small d results in a total break of the cryptosystem. To understand Wiener's attack on RSA with short secret exponents, we first need a fundamental mathematical tool: continued fractions.

### 5.1.1 Continued Fractions

#### Definition and Notation

A **simple continued fraction** (finite or infinite) is an expression of the form

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \ddots}}}$$

where $a_0 \in \mathbb{Z}$ and $a_1, a_2, a_3, \ldots$ are positive integers. For a finite continued fraction we write

$$\langle a_0; a_1, a_2, \ldots, a_m \rangle = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \cdots + \frac{1}{a_m}}}.$$

Every rational number $\frac{p}{q}$ $(q > 0)$ has a unique finite continued fraction expansion with the last partial quotient $a_m \geq 2$ (except for the trivial case $\frac{p}{q} = a_0$).

**Example:** For $\frac{41}{13}$:

$$\frac{41}{13} = 3 + \frac{2}{13} \;\Rightarrow\; a_0 = 3, \text{ remainder } \frac{2}{13}.$$

Invert $\frac{13}{2} = 6 + \frac{1}{2} \;\Rightarrow\; a_1 = 6$, remainder $\frac{1}{2}$. Invert $\frac{2}{1} = 2 \;\Rightarrow\; a_2 = 2$. Thus $\frac{41}{13} = \langle 3; 6, 2 \rangle$.

## Constructing the Continued Fraction Expansion (Algorithm)

Given a real number $x$, the **continued fraction algorithm** produces the sequence of partial quotients $a_i$.

1. Set $x_0 = x$.
2. For $i = 0, 1, 2, \ldots$:
   - $a_i = \lfloor x_i \rfloor$.
   - If $x_i = a_i$ (i.e., integer), stop.
   - Otherwise set $x_{i+1} = \dfrac{1}{x_i - a_i}$ and continue.

If $x$ is rational, the process terminates after finitely many steps; if $x$ is irrational, it continues forever.

**Example (rational):** $x = \frac{2621}{8927}$ (from Wiener's paper). $a_0 = \lfloor 2621/8927 \rfloor = 0$. Then $x_1 = 1/(2621/8927) = 8927/2621$.

$$\begin{aligned}
8927 \div 2621 &= 3 \text{ remainder } 1064 \;\Rightarrow\; a_1 = 3,\; x_2 = 2621/1064,\\
2621 \div 1064 &= 2 \text{ remainder } 493 \;\Rightarrow\; a_2 = 2,\; x_3 = 1064/493,\\
1064 \div 493 &= 2 \text{ remainder } 78 \;\Rightarrow\; a_3 = 2,\; x_4 = 493/78,\\
493 \div 78 &= 6 \text{ remainder } 25 \;\Rightarrow\; a_4 = 6,\; x_5 = 78/25,\\
78 \div 25 &= 3 \text{ remainder } 3 \;\Rightarrow\; a_5 = 3,\; x_6 = 25/3,\\
25 \div 3 &= 8 \text{ remainder } 1 \;\Rightarrow\; a_6 = 8,\; x_7 = 3/1,\\
3 \div 1 &= 3 \text{ remainder } 0 \;\Rightarrow\; a_7 = 3,\; \text{stop.}
\end{aligned}$$

Hence $\frac{2621}{8927} = \langle 0; 3, 2, 2, 6, 3, 8, 3 \rangle$.

### Reconstructing a Fraction from Its Expansion (Convergents)

Given the partial quotients $a_0, a_1, \ldots, a_m$, we compute the **convergents** $C_i = \frac{p_i}{q_i}$ (in lowest terms) using the recurrence:

$$\begin{aligned} p_{-2} &= 0, \quad p_{-1} = 1, \\ q_{-2} &= 1, \quad q_{-1} = 0, \\ p_i &= a_i p_{i-1} + p_{i-2}, \\ q_i &= a_i q_{i-1} + q_{i-2}, \qquad i = 0, 1, \ldots, m. \end{aligned}$$

Then $\frac{p_i}{q_i} = \langle a_0; a_1, \ldots, a_i \rangle$.

**Example:** For $\langle 0; 3, 2 \rangle$:

$$\begin{aligned} p_0 &= a_0 p_{-1} + p_{-2} = 0 \cdot 1 + 0 = 0, & q_0 &= a_0 q_{-1} + q_{-2} = 0 \cdot 0 + 1 = 1 & &\Rightarrow 0/1, \\ p_1 &= a_1 p_0 + p_{-1} = 3 \cdot 0 + 1 = 1, & q_1 &= a_1 q_0 + q_{-1} = 3 \cdot 1 + 0 = 3 & &\Rightarrow 1/3, \\ p_2 &= a_2 p_1 + p_0 = 2 \cdot 1 + 0 = 2, & q_2 &= a_2 q_1 + q_0 = 2 \cdot 3 + 1 = 7 & &\Rightarrow 2/7. \end{aligned}$$

Indeed $\langle 0; 3, 2 \rangle = 0 + \frac{1}{3 + \frac{1}{2}} = \frac{2}{7}$.

### Lagrange's Best Approximation Theorem and Wiener's Attack

**Theorem (Lagrange, 18th century).** Let $x$ be a real number and let $\frac{a}{b}$ be a rational number with $b > 0$. If

$$\left| x - \frac{a}{b} \right| < \frac{1}{2b^2},$$

then $\frac{a}{b}$ is a convergent of the continued fraction expansion of $x$.

*Proof.* can be found in Hardy & Wright, *An Introduction to the Theory of Numbers*, Chapter 10. The idea: if $\frac{a}{b}$ is not a convergent, then there exists a convergent $\frac{p}{q}$ with $q \leq b$ that is closer to $x$, contradicting the inequality.

## 5.1.2 Applying the Algorithm to RSA

We use the **LCM formulation** as in Wiener's original paper. Let $N = pq$, and define

$$L = \operatorname{lcm}(p - 1, q - 1).$$

The RSA exponent relation is $ed \equiv 1 \pmod{L}$, hence there exists an integer $K$ such that

$$ed = KL + 1, \qquad 1 \leq K < d. \tag{1}$$

Let $G = \gcd(p-1, q-1)$; then $L = \frac{(p-1)(q-1)}{G}$. Write $\frac{K}{G}$ in lowest terms:

$$\frac{K}{G} = \frac{k}{g}, \quad \gcd(k, g) = 1.$$

Substituting into (1) gives

$$ed = \frac{k}{g}(p-1)(q-1) + 1.$$

Multiplying by $g$:

$$edg = k(p-1)(q-1) + g. \tag{2}$$

Divide (2) by $dgN$:

$$\frac{e}{N} = \frac{k}{dg} \cdot \frac{(p-1)(q-1)}{N} + \frac{1}{dN}.$$

Since $\frac{(p-1)(q-1)}{N} = 1 - \frac{p+q-1}{N}$, we obtain

$$\frac{e}{N} - \frac{k}{dg} = -\frac{k}{dg} \cdot \frac{p+q-1}{N} + \frac{1}{dN} = \frac{-k(p+q-1)+g}{dgN}.$$

Taking absolute values,

$$\left|\frac{e}{N} - \frac{k}{dg}\right| = \frac{|k(p+q-1) - g|}{dgN}. \tag{3}$$

Assume that $d < \frac{1}{3}N^{1/4}$. Also, because $p$ and $q$ are primes of roughly equal size, $p + q - 1 < 2\sqrt{N}$. Moreover, $k < d$ (from $K < d$ and $k \leq K$) and $g$ is typically small. A careful analysis (Wiener, Section IV) shows that under these conditions

$$\left|\frac{e}{N} - \frac{k}{dg}\right| < \frac{1}{2(dg)^2}. \tag{4}$$

The fraction $\frac{k}{dg}$ is in lowest terms because $\gcd(k, g) = 1$ by construction and $\gcd(k, d) = 1$ (from $ed = KL + 1$). Therefore, by Lagrange's Theorem, $\frac{k}{dg}$ **must be a convergent** of the continued fraction expansion of $e/N$.

### Verification of a Candidate Convergent

The attacker does not know $g$ separately, only the candidate convergent $\frac{p_i}{q_i}$. Assume $\frac{p_i}{q_i} = \frac{k}{dg}$; then $q_i = dg$. To verify:

1. From (2), $e(dg) = k(p-1)(q-1) + g$. Set $w = q_i$. Compute $R = ew - 1$. If $p_i \nmid R$, discard.
2. Let $T = R/p_i$. Then $T$ should equal $(p-1)(q-1)$.
3. Compute $s = N - T + 1 = p + q$.
4. Compute discriminant $\Delta = s^2 - 4N$. If $\Delta$ is a perfect square, let $t = \sqrt{\Delta}$. Then $p = (s-t)/2$, $q = (s+t)/2$.
5. Verify $pq = N$. If true, the factorization is found. Then compute $d$ from $ed \equiv 1 \pmod{\operatorname{lcm}(p-1, q-1)}$.

Because the number of convergents is $O(\log N)$, this test runs in polynomial time.

### Example: $N = 64741,\ e = 42667$

We illustrate the attack step by step.

### Step 1: Compute continued fraction expansion of $e/N$

$x = 42667/64741 \approx 0.65885$.

$$
\begin{aligned}
a_0 &= 0,\ x_1 = 1/0.65885 \approx 1.5178,\\
a_1 &= 1,\ x_2 = 1/0.5178 \approx 1.9310,\\
a_2 &= 1,\ x_3 = 1/0.9310 \approx 1.0741,\\
a_3 &= 1,\ x_4 = 1/0.0741 \approx 13.495,\\
a_4 &= 13,\ x_5 = 1/0.495 \approx 2.0202,\\
a_5 &= 2,\ x_6 = 1/0.0202 \approx 49.5,\\
a_6 &= 49,\ x_7 = 1/0.5 = 2,\\
a_7 &= 2,\ \text{stop (remainder 0)}.
\end{aligned}
$$

Thus the expansion is $\langle 0; 1, 1, 1, 13, 2, 49, 2\rangle$.

### Step 2: Compute convergents using the recurrence

| $i$ | $a_i$ | $p_i$ | $q_i$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 |
| 2 | 1 | 1 | 2 |
| 3 | 1 | 2 | 3 |
| 4 | 13 | 27 | 41 |
| 5 | 2 | 56 | 85 |
| 6 | 49 | 2771 | 4206 |
| 7 | 2 | 5598 | 8497 |

Convergents: $C_0 = 0/1,\ C_1 = 1/1,\ C_2 = 1/2,\ C_3 = 2/3,\ C_4 = 27/41,\ C_5 = 56/85,\ C_6 = 2771/4206,\ C_7 = 5598/8497$.

### Step 3: Test each convergent as candidate $k/(dg)$

Because $N^{1/4} \approx 64741^{0.25} \approx 16$, we expect $d < 16$. Thus candidate denominators $dg$ should be small. Testing small convergents:

**Test $C_2 = 1/2$:** $k' = 1,\ w = 2$. $R = ew - 1 = 42667 \cdot 2 - 1 = 85333$. Divisible by 1, yes. $T = 85333$. $s = N - T + 1 = 64741 - 85333 + 1 = -20591$ (impossible). Discard.

**Test $C_3 = 2/3$:** $k' = 2,\ w = 3$. $R = 42667 \cdot 3 - 1 = 128000$. Divisible by 2? Yes, $128000/2 = 64000$. So $T = 64000$. $s = N - T + 1 = 64741 - 64000 + 1 = 742$. $\Delta = s^2 - 4N = 742^2 - 4 \cdot 64741 = 550564 - 258964 = 291600$. $\sqrt{291600} = 540$ (since $540^2 = 291600$). Then $p = (742 - 540)/2 = 101,\ q = (742 + 540)/2 = 641$. Check product: $101 \cdot 641 = 64741$. Success!

### Step 4: Recover $d$

We have $p = 101,\ q = 641$. Compute $L = \text{lcm}(100, 640)$. $100 = 2^2 \cdot 5^2,\ 640 = 2^7 \cdot 5$, so $L = 2^7 \cdot 5^2 = 3200$. We need $d$ such that $ed \equiv 1 \pmod{3200}$. Reduce $e$ modulo 3200: $42667 \div 3200 = 13$ remainder $42667 - 13 \cdot 3200 = 42667 - 41600 = 1067$. Solve $1067d \equiv 1 \pmod{3200}$. Note that $1067 \cdot 3 = 3201 \equiv 1 \pmod{3200}$. Hence $d = 3$.

Thus Wiener's attack successfully recovers $d = 3$, which is indeed far smaller than $N^{1/4} \approx 16$.

### 5.1.3 How to Avoid Wiener's Attack

The most direct way to defeat Wiener's attack is to ensure that the private exponent $d$ is not too small. Concretely, choosing $d$ larger than approximately $N^{1/4}$ makes the approximation error condition fail, so the continued fraction approach no longer recovers $d$. In practice, RSA implementations always use full-length $d$– that is, $d$ of the same size as the modulus $N$– which trivially satisfies $d > N^{1/4}$. Alternatively, one may keep a short $d$ but employ a very large public exponent $e$ (e.g., $e > N^{1.5}$), because the attack relies on the fraction $e/N$ being a close underestimate of $k/(dg)$. However, the most common and recommended countermeasure is to use the Chinese Remainder Theorem (CRT) for decryption: the private key consists of $p$, $q$, and the reduced exponents $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$. Even if $d$ itself is chosen full size, the CRT speeds up decryption without exposing a small $d$. Therefore, avoiding small secret exponents is simple and universally adopted.

### 5.1.4 Generalized Wiener Attacks and the Family of Wiener Attacks

Wiener's attack was the first to exploit continued fractions for small RSA secret exponents, but it only recovers $d$ when $d < N^{0.25}$. Later works extended this idea using more powerful tools, notably lattice reduction (the LLL algorithm). The most famous generalization is the **Boneh–Durfee attack** (1999), which rewrites the RSA key equation as a bivariate polynomial and applies Coppersmith's method; it succeeds when $d < N^{0.292}$. Around the same time, **Verheul and van Tilborg** (1997) proposed a variant that exhaustively searches for the most significant bits of $p+q$, extending Wiener's bound at the cost of exponential time. In 2004, **Blömer and May** improved upon both Wiener and Boneh-Durfee by constructing smaller, more efficient lattices, achieving a bound similar to $d < N^{0.29}$ while simplifying the attack. Subsequent improvements pushed the theoretical limit further: **Herrmann and May** (2009) refined the lattice construction to reach the same bound $d < N^{0.292}$ with a more straightforward proof, and **Kunihiro, Shinohara, and Izu** (2009) relaxed the conditions on the public exponent, showing that the attack works even when $e$ is not close to $N$. More generally, the family of "Wiener-like" attacks includes any method that uses rational approximation (continued fractions) or lattice-based small-inverse techniques to recover secret exponents that are unusually small. These attacks incorporate Coppersmith's theorem for finding small roots of modular equations, leading to bounds of the form $d < N^{\beta}$ where $\beta$ depends on the size of $e$ and the number of known bits. In the extreme case where the public exponent $e$ is very large (e.g., $e \approx N$), the feasible bound for $d$ can even reach $N^{0.5}$.

### 5.1.5 Wiener Attack Implementation

The following Python code is a basic implementation of Wiener's attack on RSA using continued fractions.

Wiener attack implementation

```python
import math

def get_cf(num, den):
    cf = []
    while den != 0:
        q = num // den
        cf.append(q)
        num, den = den, num - q * den
    return cf

def gen_convs(cf):
    p0, p1 = 0, 1
    q0, q1 = 1, 0
    for a in cf:
        p = a * p1 + p0
        q = a * q1 + q0
        yield p, q
        p0, p1 = p1, p
        q0, q1 = q1, q

def wiener_attack(N, e):
    cf = get_cf(e, N)
    for k, d in gen_convs(cf):
        if k == 0:
            continue
        if (e * d - 1) % k != 0:
            continue
        phi = (e * d - 1) // k
        s = N - phi + 1
        disc = s * s - 4 * N
        if disc < 0:
            continue
        t = int(math.isqrt(disc))
        if t * t != disc:
            continue
        p = (s - t) // 2
        q = (s + t) // 2
```

```
        if p * q == N:
            return d, p, q
    return None

# Example usage
if __name__ == "__main__":
    N = 8927
    e = 2621
    res = wiener_attack(N, e)
    if res:
        d, p, q = res
        print(f"d = {d}, p = {p}, q = {q}")
    else:
        print("Attack failed")
```

The Python code produces the following output:

```
d = 5, p = 113, q = 79
```

## 5.2 Low Public Exponent attacks

When a small public exponent such as $e = 3$ is used, RSA becomes vulnerable to a broadcast attack if the same message is sent to multiple recipients. This attack, due to Hastad, exploits the fact that the Chinese Remainder Theorem (CRT) can reconstruct the message once enough ciphertexts are collected.

### 5.2.1 Hastad's Broadcast Attack

Suppose a sender wants to send the same message $M$ to $k$ different parties $P_1, P_2, \ldots, P_k$. Each party has its own RSA public key $(N_i, e_i)$. For simplicity, assume all public exponents are equal to a small value $e$ (e.g., $e = 3$), and that $M$ is smaller than every modulus $N_i$. The sender encrypts $M$ with each public key and sends ciphertext $C_i = M^e \bmod N_i$ to party $P_i$.

An eavesdropping attacker collects the $k$ ciphertexts $C_1, C_2, \ldots, C_k$. The moduli $N_i$ are always coprime (otherwise the attacker could factor them). Hence, the attacker can apply the Chinese Remainder Theorem to find an integer $C_0$ such that:

$$C_0 \equiv C_i \pmod{N_i} \quad \text{for all } i = 1, \ldots, k.$$

Because each $C_i = M^e \bmod N_i$, we have $C_0 \equiv M^e \pmod{N_i}$ for every $i$. Hence $C_0 \equiv M^e \pmod{N_1 N_2 \cdots N_k}$. Since $M < N_i$ for all $i$, it follows that $M^e < N_1 N_2 \cdots N_k$. Therefore the congruence holds as an equality over the integers:

$$C_0 = M^e.$$

The attacker can then recover $M$ by computing the integer $e$-th root of $C_0$. This works as soon as the number of ciphertexts $k$ is at least $e$. For $e = 3$, three ciphertexts suffice. The attack is practical precisely because $e$ is small.

### 5.2.2 Hastad's Stronger Result (Coppersmith's method)

A naive defense against the simple broadcast attack is to pad the message differently for each recipient. For example, instead of sending the same $M$, the sender could send $M_i = i \cdot 2^m + M$ to party $P_i$ (linear padding). One might think that because the encrypted values are now different, the attack fails.

Hastad proved that this is not enough. In fact, he showed that **any fixed polynomial padding** applied to the message before encryption does **not** prevent the attack. More precisely, suppose the sender sends to party $P_i$ the encryption of $f_i(M)$, where $f_i$ is a publicly known polynomial (e.g., a linear function). As long as the number of recipients is at least the degree of the polynomial (or satisfies a certain bound), an attacker can still recover $M$ by constructing a polynomial that has $M$ as a root and then using Coppersmith's method to solve for $M$.

The attack works because the attacker knows the exact form of the padding. He can combine the $k$ ciphertexts into a system of congruences and then use algebraic techniques (root finding over integers) to recover the original message. Hastad's broadcast attack remains a serious concern whenever a small public exponent is used, even if the message is padded with different fixed values for each recipient.

We note that to properly defend against the broadcast attack above, one must use a **randomized pad** rather than a fixed one, as discussed earlier in the context of encryption padding (OAEP) and signature padding (PSS).

### 5.2.3 Franklin–Reiter Related Message Attack

When the same RSA public key encrypts two messages that are related by a known linear function, and the public exponent $e$ is small, an attacker can recover both messages using polynomial arithmetic. This attack, due to Franklin and Reiter, exploits the algebraic structure of textbook RSA and the fact that the encryption map is a permutation.

*Franklin–Reiter Theorem. Let $\langle N, e\rangle$ be an RSA public key with $N = pq$ (product of two distinct odd primes) and $\gcd(e, \phi(N)) = 1$. Suppose $M_1, M_2 \in \mathbb{Z}_N$ are two distinct messages satisfying

$$M_1 = aM_2 + b \pmod{N}$$

for known constants $a \in \mathbb{Z}_N^*$ and $b \in \mathbb{Z}_N$ with $b \neq 0$. Given the ciphertexts

$$C_1 = M_1^e \bmod N, \qquad C_2 = M_2^e \bmod N,$$

one can recover $M_1$ and $M_2$ in time polynomial in $\log N$ and $e$.

**Proof.** Because $\gcd(e, p-1) = 1$ for each prime divisor $p$ of $N$ (since $\gcd(e, \phi(N)) = 1$ and $\phi(N) = (p-1)(q-1)$), the map $x \mapsto x^e$ is a permutation of $\mathbb{Z}_p$. Indeed, for the multiplicative group $\mathbb{Z}_p^*$ of order $p-1$, consider the homomorphism $\varphi(x) = x^e$. Its kernel consists of elements satisfying $x^e = 1$. By Lagrange's theorem, the order of any such $x$ divides both $p-1$ and $e$; hence it divides $\gcd(e, p-1) = 1$. Thus the kernel is trivial, $\varphi$ is injective, and on a finite set injectivity implies bijectivity. The same holds for the other prime factor $q$. By the Chinese Remainder Theorem, $x \mapsto x^e$ is a permutation of $\mathbb{Z}_N$. Consequently, for any ciphertext $C$ there is exactly one plaintext $M$ with $M^e \equiv C \pmod{N}$; equivalently, the polynomial $x^e - C$ has exactly one linear factor $x - M$ modulo $N$.

Now define two polynomials over $\mathbb{Z}_N[x]$:

$$g_1(x) = (ax + b)^e - C_1, \qquad g_2(x) = x^e - C_2.$$

By the relation between the messages,

$$g_1(M_2) \equiv (aM_2 + b)^e - C_1 = M_1^e - C_1 \equiv 0 \pmod{N},$$

and trivially $g_2(M_2) \equiv M_2^e - C_2 \equiv 0$. Hence $x - M_2$ divides both $g_1(x)$ and $g_2(x)$ in $\mathbb{Z}_N[x]$.

Because the equation $x^e = C_2$ has the unique solution $x = M_2$, the polynomial $g_2(x)$ factors

as

$$g_2(x) = (x - M_2) \cdot q(x),$$

where $q(x)$ has no linear factor (otherwise a second distinct root would exist, contradicting uniqueness). The polynomial $g_1(x)$ shares the factor $x - M_2$ but is not a multiple of $g_2(x)$ (that would imply $M_1 = M_2$, which is false since $b \neq 0$). Therefore the greatest common divisor of $g_1(x)$ and $g_2(x)$ is exactly $x - M_2$ (up to multiplication by a unit in $\mathbb{Z}_N$). Computing this gcd using the Euclidean algorithm for polynomials yields the linear factor, from which $M_2$ is read directly. Then $M_1 = aM_2 + b \bmod N$ is recovered. *Example

Take $p = 47$, $q = 59$; then $N = 2773$. Choose $e = 3$ and the linear relation $M_1 = 2M_2 + 5$ with $M_2 = 1000$. Then $M_1 = 2005$. Compute the ciphertexts:

$$C_1 = 2005^3 \bmod 2773 = 2356, \qquad C_2 = 1000^3 \bmod 2773 = 740.$$

The attacker constructs

$$g_1(x) = (2x + 5)^3 - 2356, \qquad g_2(x) = x^3 - 740$$

over $\mathbb{Z}_{2773}[x]$. The Euclidean algorithm gives $\gcd(g_1, g_2) = x - 1000$, revealing $M_2 = 1000$ and then $M_1 = 2 \cdot 1000 + 5 = 2005$.

The attack requires computing the greatest common divisor of two polynomials of degree $e$. Using the Euclidean algorithm, this takes $O(e^2 \log N)$ operations. For small $e$ (e.g., $e = 3$ or even $e = 65537$), this is entirely practical. However, if $e$ is very large (comparable to $N$), the polynomial gcd becomes infeasible. Moreover, the attack critically depends on the linear relation being exactly known and on the absence of padding. If the plaintexts are not related by a low-degree polynomial, or if a randomized padding scheme is used, the attack fails.

## Countermeasures.

To prevent the Franklin–Reiter attack, one must never encrypt two related messages using textbook RSA with the same small public exponent. Proper RSA encryption mandates a randomized padding scheme such as OAEP (Optimal Asymmetric Encryption Padding). OAEP destroys any algebraic relation between plaintexts because the padded message is a pseudorandom function of the original plaintext and a fresh random seed. Consequently, even if the underlying messages are linearly related, the padded values are not, and the attack no longer applies. The same principle holds for RSA signatures, where randomized padding like PSS is essential.

### 5.2.4 Partial Key Exposure Attack

Suppose an attacker is able to obtain only a fraction of the bits of the RSA private exponent $d$ – for example, through a side-channel attack. The question is whether this partial information suffices to reconstruct the entire private key. When the public exponent $e$ is small (specifically $e < \sqrt{N}$), it turns out that knowing just **one quarter** of the bits of $d$ (the least significant bits) allows the attacker to recover the full $d$ in polynomial time. This is the partial key exposure attack of Boneh, Durfee, and Frankel. Moreover, when $e$ is extremely small (e.g., $e = 3$), the RSA equation directly **leaks half of the most significant bits** of $d$ without any side-channel – a fact we explain at the end of the section.

**theorem: (Coppersmith, 1996)** Let $N = pq$ be an $n$-bit RSA modulus (so $p$ and $q$ have about $n/2$ bits). If one knows either the $n/4$ least significant bits of $p$ or the $n/4$ most significant bits of $p$, then one can factor $N$ in time polynomial in $n$.

*Remark.* Since $p$ has $n/2$ bits, $n/4$ bits of $N$ correspond to exactly one half of the bits of $p$. Coppersmith's proof uses lattice reduction (LLL) to find the unknown half.

**theorem: (Boneh, Durfee, Frankel, 1998 – LSB version)** Let $\langle N, d\rangle$ be an RSA private key with $N$ an $n$-bit modulus and let the public exponent $e$ satisfy $e < \sqrt{N}$. Given the $n/4$ least significant bits of $d$, the attacker can reconstruct all of $d$ in time $O(e \log e)$ times a polynomial in $n$.

*Proof.* From the RSA key equation there exists an integer $k$ with $1 \le k \le e$ such that

$$ed = 1 + k\,\varphi(N), \qquad \varphi(N) = N - p - q + 1. \tag{1}$$

Substituting $q = N/p$ and multiplying by $p$ gives

$$ed\,p = p + k\Big(Np - p^2 - N + p\Big).$$

Let $M = 2^{n/4}$. the attacker knows $d \bmod M$, hence he knows $ed \bmod M$. Reduce (2) modulo $M$:

$$(ed)p \equiv p + k(Np - p^2 - N + p) \pmod{M}.$$

Simplifying the right-hand side:

$$p + kNp - kp^2 - kN + kp = -kp^2 + (1 + kN + k)p - kN.$$

Thus we obtain the quadratic congruence in the unknown prime $p$:

$$kp^2 - \Big((ed) - (1 + kN + k)\Big)p + kN \equiv 0 \pmod{M}.$$

For each possible $k = 1, 2, \ldots, e$, he solves the quadratic equation in p and obtains a number of candidate values for $p \pmod{M}$.

For each candidate $p$, the attacker invokes Coppersmith's theorem: because he now knows the low half of $p$, the theorem factors $N$ in polynomial time. At least one candidate (the one coming from the true $k$) will succeed. Therefore after some attempts he obtains the factorization $(p, q)$. He then computes $\varphi(N) = (p-1)(q-1)$ and recovers $d = e^{-1} \bmod \varphi(N)$. The total running time is $O(e \log e) \cdot \text{poly}(n)$. □

For very small $e$ (e.g., $e = 3$), the RSA equation directly reveals half the most significant bits of $d$ without any side-channel. From (1) we have:

$$ed = 1 + k(N - p - q + 1) = kN + 1 - k(p + q - 1).$$

Hence

$$d = \frac{kN + 1}{e} - \frac{k(p + q - 1)}{e}.$$

Define $\hat{d} = \left\lfloor \frac{kN+1}{e} \right\rfloor$. Because $k(p + q - 1) < 2k\sqrt{N}$ and $k \leq e$, then

$$|\hat{d} - d| \leq \frac{k(p + q)}{e} \leq \frac{3k\sqrt{N}}{e} < 3\sqrt{N}.$$

Therefore, $\hat{d}$ serves as a close approximation to $d$. The derived bound implies that for the majority of private exponents $d$, the most significant half of the bits of $\hat{d}$ coincide with those of $d$. Because only $e$ distinct values of $k$ are possible, Marvin can form a small set of size at most $e$ that contains one element equal to the most significant half of the bits of $d$. The case $e = 3$ is particularly noteworthy: one can prove that $k = 2$ always holds, and consequently the RSA system fully reveals half of the most significant bits of the private exponent $d$.

## 5.3 Homomorphic Attacks

The "textbook" version of RSA – where encryption and decryption are applied directly to integers representing messages – suffers from a fundamental algebraic weakness: **multiplicative homomorphism**.

### Multiplicative Homomorphism

For a cryptosystem with encryption function $E$ and decryption function $D$, we say it is multiplicatively homomorphic if for all messages $m_1, m_2$ in the message space

$$D\Big(E(m_1) \cdot E(m_2)\Big) = m_1 \cdot m_2 \qquad (\text{or } \equiv m_1 m_2 \pmod{N}).$$

We have $E(m) = m^e \bmod N$ and $D(c) = c^d \bmod N$. Hence

$$D\Big(E(m_1) \cdot E(m_2)\Big) = (m_1^e m_2^e)^d = (m_1 m_2)^{ed} = m_1 m_2 \pmod{N}.$$

The same property holds for signatures: $S(m) = m^d \bmod N$ satisfies

$$S(m_1) \cdot S(m_2) \equiv S(m_1 m_2) \pmod{N}.$$

## 5.3.1 Common Modulus Attacks on RSA

This property allows an attacker to combine ciphertexts (or signatures) without knowing the private key.

#### A) Simmons' Attack

In RSA, for a user with modulus $N = pq$ and a private exponent $d$ satisfying $ed \equiv 1 \pmod{\varphi(N)}$, there exists an integer $k \geq 1$ such that

$$ed - 1 = k \cdot \varphi(N).$$

Thus $m = ed - 1$ is a **known multiple** of $\varphi(N)$. For any integer $a$ coprime to $N$, Euler's theorem gives $a^{\varphi(N)} \equiv 1 \pmod{N}$, and raising to the power $k$ yields

$$a^m \equiv 1 \pmod{N}.$$

Now, $m$ is even because $\varphi(N)$ is even for odd primes $p, q$. Write $m = 2^s \cdot t$ with $t$ odd. Define the sequence

$$x_0 = a^t \bmod N, \quad x_{i+1} = x_i^2 \bmod N \ \ (i \geq 0).$$

Since $a^m = a^{2^s t} \equiv 1 \pmod{N}$, we have $x_s \equiv 1$. Let $i$ be the smallest index such that $x_{i+1} \equiv 1 \pmod{N}$. Then $x_i^2 \equiv 1$ but $x_i \not\equiv 1$. If also $x_i \not\equiv -1$, then $x_i$ is a nontrivial square root of 1 modulo $N$:

$$(x_i - 1)(x_i + 1) \equiv 0 \pmod{N},$$

and neither factor is divisible by $N$. Hence $\gcd(x_i - 1, N)$ is a proper divisor (either $p$ or $q$). If we encounter $x_i \equiv -1$, we simply pick another random $a$ and repeat. The probability of success per trial is at least $1/2$. Once a factor is found, RSA is broken.

**Algorithm**

1. Compute $s = v_2(m)$, $t = m/2^s$.
2. Choose random $a \in [2, N-1]$. If $\gcd(a, N) \neq 1$, return it.
3. Compute $x_0 = a^t \bmod N$.
4. For $i = 0$ to $s - 1$:
   - $x_{i+1} = x_i^2 \bmod N$.
   - If $x_{i+1} \equiv 1$ and $x_i \not\equiv \pm 1$, return $\gcd(x_i - 1, N)$.
5. Otherwise go to step 2.

**Example**

Let $p = 11, q = 17$, so $N = 187$. Choose $e = 7$; then $d = 23$ because $7 \cdot 23 = 161 = 1 + 1 \cdot 160$ . We have also $m = ed - 1 = 160$. Write $160 = 2^5 \cdot 5$ ($s = 5, t = 5$). Pick $a = 2$ (coprime to 187). Compute:

$$\begin{aligned} x_0 &= 2^5 = 32 \mod 187, \\ x_1 &= 32^2 = 1024 \equiv 89, \\ x_2 &= 89^2 = 7921 \equiv 67, \\ x_3 &= 67^2 = 4489 \equiv 1. \end{aligned}$$

Here $i = 2$ because $x_3 = 1$. $x_2 = 67 \not\equiv \pm 1$. Then $\gcd(67 - 1, 187) = \gcd(66, 187) = 11$.

### Implementation of the attack

```python
import math
import random

def factorize_from_multiple(N, m):

    s = 0
    t = m
    while t % 2 == 0:
        t //= 2
        s += 1

    # Try random bases a
    for _ in range(20):  # enough attempts for high probability
        a = random.randrange(2, N)
        g = math.gcd(a, N)
        if g != 1:
            # a shares a factor with N  we already have a factor
            return g

        x = pow(a, t, N)

        if x == 1 or x == N-1:
            continue

        for _ in range(s):
            prev = x
            x = pow(x, 2, N)
            if x == 1:
                if prev != N-1:
                    return math.gcd(prev - 1, N)
                break
            if x == N-1:
                break

    raise ValueError("Failed to factor N; check that m is a multiple of
       phi(N)")

if __name__ == "__main__":
    N = 187
    m = 160   # = e*d - 1 (with e=7, d=23)
    factor = factorize_from_multiple(N, m)
    print(f"The first factor p = {factor}")
```

```
42      print(f"The second factor q = {N // factor}")
```

```
The first factor p = 11
The second factor q = 17
```

### B) Recovering a Message from Two Ciphertexts

The same plaintext $m$ (with $0 < m < N$) is encrypted for two different users with public exponents $e_1, e_2$ (both coprime to $\phi(N)$). The ciphertexts are

$$c_1 = m^{e_1} \bmod N, \qquad c_2 = m^{e_2} \bmod N.$$

The attacker captures $c_1$ and $c_2$. If $\gcd(e_1, e_2) = 1$, he can recover $m$ without factoring $N$. Because $\gcd(e_1, e_2) = 1$, there exist integers $a, b$ (computed via the extended Euclidean algorithm) such that

$$ae_1 + be_2 = 1.$$

Now compute

$$c_1^a \cdot c_2^b \equiv (m^{e_1})^a \cdot (m^{e_2})^b = m^{ae_1+be_2} = m^1 = m \pmod N.$$

If $a$ or $b$ is negative, we need to work with modular inverses. For instance, if $a < 0$, we compute the modular inverse of $c_1$ modulo $N$ (which exists because $\gcd(c_1, N) = 1$, otherwise $N$ would be factored). The result is still $m$.

### Conclusion

The common modulus attacks demonstrate that sharing the same RSA modulus among multiple users is catastrophic: any user can factor the modulus using their own key pair, thereby recovering all other private keys, and even without any private key, two ciphertexts of the same message under coprime exponents directly leak the plaintext. Hence, RSA moduli must never be reused.

### 5.3.2 Chosen cipher text

Among the most devastating consequences of RSA's multiplicative homomorphism is its vulnerability to **chosen ciphertext attacks**. In such an attack, the adversary has temporary access to a decryption oracle – a device that decrypts arbitrary ciphertexts (except the target) and returns the corresponding plaintext. The attacker's goal is to decrypt a specific target ciphertext $c = m^e \bmod N$ without knowing the private key $d$.

The attack proceeds as follows:

1. **Target:** The attacker intercepts a ciphertext $c = m^e \bmod N$ intended for the victim.
2. **Blinding:** The attacker chooses a random integer $r$ with $\gcd(r, N) = 1$ and computes a modified ciphertext
$$c' = c \cdot r^e \bmod N.$$
Note that $c' \neq c$ (unless $r = 1$), so the decryption oracle may accept it.
3. **Oracle query:** The attacker submits $c'$ to the decryption oracle. The oracle returns
$$m' = D(c') = (m^e \cdot r^e)^d = m^{ed} \cdot r^{ed} = m \cdot r \pmod N.$$
4. **Recovery:** Because the attacker knows $r$, they compute $r^{-1} \bmod N$ and obtain
$$m = m' \cdot r^{-1} \pmod N.$$

The decryption oracle never sees the original ciphertext $c$, only a blinded version, yet the homomorphic property allows the attacker to extract $m$ directly. This demonstrates that textbook RSA is completely insecure in the presence of even a single decryption query.

### 5.3.3 Davida's Attack (Chosen-Message Universal Forgery)

Davida's attack exploits the multiplicative homomorphism of textbook RSA signatures to forge a valid signature for any target message after obtaining the signatures of two specially chosen messages. This is a **chosen-message attack** leading to **universal forgery**: the adversary can sign any message of their choice without knowing the private key.

The attack proceeds as follows:

1. **Target:** The attacker wants a valid signature $S(m) = m^d \bmod N$ for a chosen message

$m$ (with $\gcd(m, N) = 1$).

2. **Choose blinding factors:** The attacker selects a random integer $r$ with $\gcd(r, N) = 1$ and constructs two messages:

$$m_1 = m \cdot r \bmod N, \qquad m_2 = r^{-1} \bmod N.$$

3. **Obtain signatures:** The attacker requests signatures from the legitimate signer for $m_1$ and $m_2$. The signer returns

$$S_1 = m_1^d \bmod N, \qquad S_2 = m_2^d \bmod N.$$

4. **Forge the target signature:** The attacker computes

$$S = S_1 \cdot S_2 \bmod N.$$

Then $S$ is a valid signature for the original message $m$.

The attacker never needs to know the private key; only two legitimate signatures on chosen (but seemingly harmless) messages are required.

### 5.3.4 LSB Oracle Attack on RSA

The LSB (least significant bit) oracle attack on RSA exploits the fact that knowing just the parity (even or odd) of the decrypted plaintext allows an attacker to iteratively narrow down the plaintext value until it is fully recovered. This is a side-channel attack that relies purely on parity leakage, distinct from padding oracle attacks.

To understand the attack, first recall the meaning of the least significant bit. In binary representation, any integer's LSB is 0 if the number is even and 1 if it is odd. So an oracle that returns the LSB of a decrypted plaintext is simply telling us whether that plaintext is even or odd. On its own this seems trivial, but when combined with clever ciphertext manipulation it becomes a powerful tool.

Consider standard RSA: the modulus $N = p \cdot q$ is odd (both primes are odd), the public key is $(e, N)$, the private key is $d$, and a ciphertext is $c = m^e \bmod N$ where $m$ is the plaintext. The attacker is assumed to have access to an oracle that, given any ciphertext, decrypts it and returns only the LSB of the resulting plaintext.

The core idea is to multiply the ciphertext by $2^e \bmod N$ and send the result to the oracle. Because of RSA's homomorphic property, decrypting $c \cdot 2^e \bmod N$ yields

$$m' = 2m \bmod N.$$

The oracle then tells us whether $m'$ is even or odd. Now, how does the parity of $2m \bmod N$ reveal information about the original $m$? There are two cases:

- If $m < N/2$, then $2m$ is still less than $N$ (since $2m < N$). Therefore $2m \bmod N = 2m$, which is even. The oracle returns "even".
- If $m \geq N/2$, then $2m \geq N$. In this case $2m \bmod N = 2m - N$. Since $N$ is odd, subtracting an odd number from an even number ($2m$ is even) gives an odd result. The oracle returns "odd".

Thus, the parity of $2m \bmod N$ tells us exactly whether $m$ lies in the lower half $[0, N/2)$ (even $\rightarrow$ lower half) or the upper half $[N/2, N)$ (odd $\rightarrow$ upper half). In other words, a single LSB query reveals the most significant bit (MSB) of the plaintext: the MSB is 0 for the lower half and 1 for the upper half.

### Algorithm

The attack proceeds iteratively. We maintain an interval $[L, R)$ that is guaranteed to contain the secret plaintext $m$. Initially, $L = 0$ and $R = N$. Each step uses one oracle query to halve the interval.

1. **Set up the initial interval.**
   Let $L = 0$ and $R = N$.
   *The plaintext satisfies* $0 \leq m < N$.

2. **For** $k = 1, 2, 3, \ldots$ **until the interval contains only one integer:**

   (a) **Construct a manipulated ciphertext.**
   Compute $c_{\text{query}} = c \cdot (2^k)^e \bmod N$.
   *This is equivalent to encrypting the value* $2^k m$ *because*
   $c \cdot (2^k)^e \equiv m^e \cdot (2^k)^e \equiv (2^k m)^e \pmod N$.

   (b) **Query the LSB oracle.**
   Send $c_{\text{query}}$ to the oracle. The oracle decrypts and returns the LSB of $2^k m \bmod N$.

(c) **Interpret the answer to halve the interval.**

- If the oracle returns **even**, then $2^k m \bmod N$ is even. This implies that the current scaled plaintext lies in the lower half of the modulus. Consequently, the original $m$ belongs to the left half of $[L, R)$. Update:

$$R \leftarrow \frac{L+R}{2}.$$

- If the oracle returns **odd**, then $2^k m \bmod N$ is odd. This implies that the current scaled plaintext lies in the upper half of the modulus. Hence, $m$ belongs to the right half of $[L, R)$. Update:

$$L \leftarrow \frac{L+R}{2}.$$

*The parity of $2^k m \bmod N$ tells us whether $2^{k-1} m \bmod N$ (or, after renormalisation, the previously scaled value) is less than $N/2$ or not. This is exactly the binary search decision.*

(d) **Check for termination.**
If $R - L = 1$ (i.e., the interval contains exactly one integer), then that integer is $m$. Stop.

3. **Output the recovered plaintext.**
The unique integer in the final interval is the secret $m$.

Mathematically, each query provides exactly one bit of information about $m$. Because $m$ lies in $[0, N)$ and can be represented with at most $\lceil \log_2 N \rceil$ bits, the total number of queries required is $O(\log N)$. Each query involves a modular exponentiation (to compute the manipulated ciphertext) and a call to the oracle. The attack is efficient and devastating.

**Example**

let $N = 101$ and the target plaintext be $m = 73$. Start with $[0, 101)$. Query with $2m \bmod N$: the oracle returns odd, so $m \in [51, 101)$. Next query with $4m \bmod N$: the oracle returns even, so $m \in [51, 76)$. Next query with $8m \bmod N$: the oracle returns odd, so $m \in [63, 76)$. Continuing this process, the interval collapses to $\{73\}$. No private key is ever needed; only the parity answers.

The LSB oracle, despite leaking only a single parity bit, effectively reveals the entire plaintext because that bit can be reinterpreted as a decision about which half of the remaining interval contains $m$.

### Conclusion

These homomorphic attacks succeed because RSA's multiplicative property allows ciphertext manipulation to leak plaintext information. Padding schemes like OAEP break this algebraic malleability by adding structured randomness before encryption, ensuring that any tampered ciphertext decrypts to an invalid format and is rejected before any bit is revealed. Thus, proper padding is the essential defense against such attacks. In the next section we will discus padding.

## 5.4 Padding in RSA

Textbook RSA—the raw operation $m \mapsto m^e \bmod N$—is mathematically elegant but cryptographically brittle. In isolation, it suffers from deterministic encryption, malleability, small-message exposure, and signature forgery. Padding resolves all these issues by wrapping the message (or its hash) in a structured, often randomized, full-length envelope before the RSA operation.

### 5.4.1 Why Padding Is Necessary

Without padding, an attacker can exploit the following weaknesses:

- **Deterministic encryption**: Identical plaintexts produce identical ciphertexts, enabling traffic analysis and dictionary attacks.
- **Malleability**: Multiplying a ciphertext by $2^e$ multiplies the plaintext by 2, which leads to adaptive chosen-ciphertext attacks (e.g., the LSB oracle attack).
- **Small plaintext space**: If the plaintext is very small (e.g., a symmetric key), an attacker can compute an integer $e$-th root to recover it directly.
- **Signature existential forgery**: Lacking a fixed format, an adversary can combine valid signatures or produce a signature for an arbitrary message without the private key.

Padding defeats each of these attacks.

### 5.4.2 What is a Hash Function?

A **hash function** takes an input of arbitrary length (a message, file, or any data) and produces a fixed-length output called a **hash value** or **digest**. It is designed to be **one-way** (given the hash, it is infeasible to recover the original input) and **collision-resistant** (it is hard to find two different inputs that produce the same hash). Hash functions are used in RSA signatures to reduce the message to a fixed size before signing.

**Example**: SHA-256 (Secure Hash Algorithm 256-bit). The message `"Hello"` hashes to:

`185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969`

(in hexadecimal). Changing even one character in the input produces a completely different, unpredictable hash.

### 5.4.3 How Padding Works: Common Principles

All padding schemes share three core functions:

1. **Expansion** – The input (message or hash) is extended to the full length of the RSA modulus (e.g., 2048 bits), eliminating any possibility of small-integer attacks.
2. **Randomization** – A random value is mixed into the padded block. For encryption this is called a **seed**; for signatures it is called a **salt**. Randomization ensures that even identical inputs yield different padded blocks.
3. **Integrity marker** – A fixed, predictable byte pattern (e.g., `0x01` or a sequence of zero bytes) is embedded at a known location. After decryption, the receiver verifies this marker; if it is absent or corrupted, the ciphertext (or signature) is rejected.

These three principles are instantiated differently for encryption and for signatures.

### 5.4.4 Padding for Encryption: OAEP

**Key term – seed**: A seed is a random bit string generated freshly for each encryption operation. It is used to mask the message so that the output appears uniformly random.

**General operation**:

- The message is padded with a fixed marker (e.g., `0x00...001`).
- A random **seed** is generated.
- The seed is used to mask the message block (via a mask generation function).
- The message block is then used to mask the seed itself.
- The final padded block consists of the masked seed concatenated with the masked message.

**Decryption and verification**: After RSA decryption, the receiver reverses the masking using the same functions, extracts the message, and checks the integrity marker. If the marker is not exactly as expected, the ciphertext is rejected.

**Why it is secure**:

- **Randomization** via the seed destroys determinism; the same message encrypts to different ciphertexts each time.
- **Integrity marker** detects any tampering; modified ciphertexts almost never produce a valid marker, so the receiver rejects them without leaking useful information.
- **Full-length expansion** ensures that the plaintext is never small enough for an $e$-th root attack.

### 5.4.5 Padding for Signatures

Two principal schemes exist: the legacy PKCS#1 v1.5 and the modern PSS (Probabilistic Signature Scheme).

#### Legacy scheme: PKCS#1 v1.5

**General operation**:

- Compute the cryptographic hash of the message (using a function like SHA-256).
- Construct a fixed template: `0x00 0x01`, followed by a string of `0xFF` bytes, then a `0x00` separator, then an algorithm identifier and the hash itself.
- This template is the padded block. It contains no randomness.

**Verification**: The verifier recovers the padded block from the signature, checks the exact structure (`00 01 ... FF ... 00 ...`), extracts the hash, and compares it with the hash of the received message.

**Security limitation**: Because there is no salt, the scheme is deterministic and has known weaknesses. It is no longer recommended for new applications.

#### Modern scheme: PSS (Probabilistic Signature Scheme)

**Key term – salt**: A salt is a random value (typically of the same length as the hash output) generated freshly for each signature. It introduces unpredictability and prevents certain forgery attacks.

**General operation**:

- Compute the hash of the message.
- Generate a random **salt**.
- Form a new string that concatenates a fixed prefix, the message hash, and the salt. Hash this string to obtain a second hash value.
- Construct a data block containing the salt and a fixed marker.
- Mask this data block using the second hash.
- The final padded block is the second hash, the masked data, and a fixed trailer byte.

**Verification**: The verifier recovers the second hash and the masked data, unmask to obtain the salt, recompute the second hash from the message hash and the salt, and confirm equality.

**Why it is stronger**:

- **Salt** makes signatures probabilistic: signing the same message twice produces different output, defeating forgery strategies that rely on deterministic collisions.
- **Double hashing** provides domain separation and strengthens the security reduction.
- **Provable security**: PSS is proven existentially unforgeable under adaptive chosen-message attacks (assuming the RSA problem is hard).

### 5.4.6 How Padding Protects Against Core Vulnerabilities

- **Deterministic encryption**: Seed (for encryption) or salt (for signatures) ensures that identical inputs yield distinct outputs. Attackers cannot build dictionaries or detect repetitions.
- **Malleability**: The integrity marker embedded in the padded block acts as a checksum. Any modification to the ciphertext corrupts this marker with overwhelming probability, causing the receiver to reject the message. The attacker learns nothing, and the attack chain is broken.
- **Small message exposure**: Padding expands the input to the full length of $N$. The resulting integer is no longer small, making $e$-th root attacks impossible.
- **Signature forgery**: Rigid formatting (v1.5) or salt-based double hashing (PSS) destroys the multiplicative homomorphism of raw RSA. Multiplying valid signatures does not produce a valid padded block, nor can an attacker compute a signature from an arbitrary integer.

### 5.4.7 Conclusion

Padding is not an optional enhancement; it is a mandatory security layer that transforms fragile textbook RSA into a robust cryptographic primitive. For encryption, OAEP employs a random **seed** and an integrity marker to achieve chosen-ciphertext security. For signatures, PSS uses a random **salt** and a two-layer hash construction to provide existential unforgeability. In both cases, the fundamental principles are the same: expand the input, randomize it, and embed a verifiable pattern. These simple measures defeat every major practical attack on RSA.

## 5.5 Implementation Attacks

### 5.5.1 Timing and Power Attacks on RSA

The standard method for RSA private-key operation (decryption or signature generation) is the square-and-multiply algorithm, which processes the bits of the private exponent $d$ from most significant to least significant. At each iteration, a squaring is always performed, followed by a multiplication by the base $M$ only if the current exponent bit is 1. This conditional multiplication creates a direct dependency between the bits of $d$ and both the

execution time and the power consumption: a multiply step takes slightly longer and consumes a different amount of power than a squaring. In a **timing attack**, an adversary measures the total decryption time for many carefully chosen inputs $M$ and uses statistical tests (e.g., correlation or t-tests) to decide whether a particular bit of $d$ is 0 or 1. Because the time difference between a square-only and a square-then-multiply iteration is tiny but measurable, averaging over thousands of trials allows the attacker to reconstruct $d$ bit by bit.

refines this idea by monitoring the device's instantaneous power consumption during a single exponentiation. In Simple Power Analysis (SPA), the attacker inspects one power trace and directly identifies the sequence of square and multiply operations, thereby reading the bits of $d$ from the trace. Differential Power Analysis (DPA) is more powerful: it records many traces for different random inputs and uses statistical hypothesis testing (e.g., correlation with predicted intermediate values) to extract the secret bits even when the signal is noisy and when the algorithm uses countermeasures like dummy multiplications. The core reason power analysis works is that the power consumed by a CMOS chip (Complementary Metal-Oxide-Semiconductor, the technology used in most integrated circuits) depends on the Hamming weight (number of 1-bits) of the data being processed; the intermediate values in square-and-multiply are functions of $M$ and the already recovered bits of $d$, so an attacker can predict those values and correlate them with actual power measurements.

Consequently, both timing and power attacks demonstrate that the square-and-multiply algorithm, if implemented without side-channel countermeasures, directly leaks the private exponent $d$. The most effective protections are exponent blinding (randomizing $d$ before each exponentiation by adding a random multiple of $\varphi(N)$ and base blinding (multiplying the input $M$ by a random factor). These measures break the statistical dependency between the measurements and the secret bits, making the attacks infeasible. Modern secure RSA libraries always include such countermeasures.

### 5.5.2 Fault-Based Attack on RSA-CRT

RSA with the Chinese Remainder Theorem (CRT) speeds up signing. Let us recall that in order To sign a message $x$ with secret exponent $d$ and modulus $N = p \cdot q$ (two large primes):

1. Compute $S_1 = x^d \bmod p$.
2. Compute $S_2 = x^d \bmod q$.
3. Combine $S_1$ and $S_2$ via CRT to obtain the correct signature $S = x^d \bmod N$.

The attack exploits a hardware fault that occurs during the computation of **only one** of the two branches.

An attacker asks the black-box to sign a known message $x$. Due to a random fault (e.g., a bit flip) during the computation of $S_1$ modulo $p$, the box outputs a faulty signature $\hat{S}$. The computation modulo $q$ remains correct. Consequently:

- $\hat{S} \equiv S \pmod{q}$ (they agree modulo $q$),
- $\hat{S} \not\equiv S \pmod{p}$ (they disagree modulo $p$).

The difference $S - \hat{S}$ is divisible by $q$ (because they are equal modulo $q$) but **not** divisible by $p$. Therefore the greatest common divisor of $S - \hat{S}$ and $N$ must be exactly $q$:

$$\gcd(S - \hat{S},\, N) = q.$$

Once $q$ is known, $p = N/q$ follows immediately. Factoring $N$ completely breaks the RSA system – the private exponent $d$ can be recomputed.

**Lenstra's improvement (even simpler).** The paper also cites a later improvement by Lenstra: the correct signature $S$ is not needed. Using only the faulty signature $\hat{S}$ and the known message $x$:

- Compute $\hat{S}^e \bmod N$ where $e$ is the public exponent.
- Because of the fault, $x \equiv \hat{S}^e \pmod{q}$ but $x \not\equiv \hat{S}^e \pmod{p}$.
- Hence $\gcd(x - \hat{S}^e, N) = q$.

Thus a **single** interaction that yields one faulty signature is sufficient to factor the modulus and recover the entire private key.

### Countermeasures

To prevent fault attacks, implementations must detect errors before outputting the result. Common countermeasures include:

- **Verification with the public exponent:** After computing $S$, check that $S^e \bmod N$ equals the original message $M$. This detects any fault that altered the result, but it doubles the computation time.

- **Redundant computation:** Compute the private operation twice (possibly using different algorithms) and compare outputs.
- **Secure CRT recombination:** Include a check that the recombined result indeed is congruent to $S_p$ modulo $p$ and to $S_q$ modulo $q$. If not, abort.
- **Randomisation (blinding):** Multiply the message by a random factor before exponentiation and correct afterwards. A fault will likely destroy the blinding relationship, causing the verification to fail.
- **Error-detecting codes:** Encode intermediate values with checksums that are verified at the end.

### 5.5.3 Bleichenbacher's Chosen Ciphertext Attack on RSA

Bleichenbacher's attack (1998) is a landmark chosen ciphertext attack against RSA when used with the PKCS #1 v1.5 padding scheme. It demonstrates that even a single bit of information from a decryption oracle—whether the padding is valid—can completely break the cryptosystem. This attack was successfully used against early SSL/TLS implementations, and it motivated the development of provably secure padding schemes such as OAEP.

#### PKCS #1 v1.5 Padding (Recall)

For the encryption to be secure, the plaintext must be structured. PKCS #1 v1.5 defines a specific format for the encoded message before RSA encryption. For an $n$ of $k$ bytes (e.g., 128 bytes for 1024-bit RSA), the encoded block has the form:

$$\texttt{0x00} \parallel \texttt{0x02} \parallel \text{PS} \parallel \texttt{0x00} \parallel \text{DATA}$$

where:

- `0x00` and `0x02` are fixed headers.
- `PS` is a non-zero random padding string of at least 8 bytes.
- `0x00` separates the padding from the actual data.

When decrypting a ciphertext, the recipient checks whether the resulting plaintext starts with `0x00 0x02` and contains the expected separator. If the format is correct, the decryption proceeds; otherwise an error (often "invalid padding") is returned.

**The Padding Oracle:** A *padding oracle* is any system that, given a ciphertext $c$, decrypts it and indicates whether the obtained plaintext has valid PKCS #1 v1.5 formatting. The attacker does not need the actual plaintext—only the binary answer: "valid" or "invalid". Such oracles can arise from explicit error messages, timing differences, or other side channels.

### The Million-Message Narrowing Attack

Bleichenbacher's attack uses the oracle to gradually narrow down the set of possible plaintext values for a target ciphertext $c$. The ultimate goal is to recover $m = c^d \bmod n$.

For any integer $s$, consider the modified ciphertext:

$$c' = c \cdot s^e \bmod n.$$

Because of the homomorphic property of RSA, decrypting $c'$ yields:

$$m' = (c')^d \bmod n = (c^d \cdot (s^e)^d) \bmod n = m \cdot s \bmod n.$$

The attacker chooses different values of $s$ and asks the oracle whether the decryption of $c'$ (i.e., $m \cdot s \bmod n$) has valid PKCS #1 v1.5 padding. The answer reveals whether $m \cdot s \bmod n$ lies in a certain interval of integers that have the correct leading bytes.

To express this condition precisely, let $B = 2^{8(k-2)}$ (for a modulus of $k$ bytes). The condition for valid PKCS #1 v1.5 padding can be expressed as:

$$2B \leq \text{plaintext} \leq 3B - 1$$

because the first two bytes are `0x00 0x02`, which corresponds to a numeric value between $2B$ and $3B - 1$. More precisely, the valid plaintext lies in the interval

$$[2B,\ 3B - 1].$$

For a given ciphertext $c$, let $M_0 = [2B,\ 3B - 1]$ be the initial interval of possible plaintexts.

With this mathematical formulation in place, the attack proceeds iteratively as follows:

The attacker multiplies the target ciphertext by $s_0^e$ for some random $s_0$ and queries the oracle until a valid padding is found. This yields a first multiplier $s_0$ and an interval $M_0$.

Once an interval $M_{i-1} = [a, b]$ is known, the attacker searches for the smallest integer $s_i > s_{i-1}$ such that the plaintext $m \cdot s_i \bmod n$ falls into the valid padding interval. Because $m \cdot s_i \bmod n$ is actually $m \cdot s_i$ (if no wrap-around) or $m \cdot s_i - n$ (if it exceeds $n$), the condition becomes:

$$2B \leq m \cdot s_i - tn \leq 3B - 1 \quad \text{for some integer } t.$$

This can be rearranged into constraints on $m$:

$$\frac{2B + tn}{s_i} \leq m \leq \frac{3B - 1 + tn}{s_i}.$$

Thus, from each $s_i$ that yields a valid oracle response, the attacker computes a new interval for $m$ and intersects it with the previous interval $M_{i-1}$. The new interval is:

$$M_i = M_{i-1} \cap \bigcup_t \left[ \frac{2B + tn}{s_i},\ \frac{3B - 1 + tn}{s_i} \right].$$

After each iteration, the interval length shrinks. Once $M_i$ contains only one integer, that integer is the original plaintext $m$.

The core lemma is that if the oracle returns "valid" for a ciphertext $c' = c \cdot s^e \bmod n$, then $m$ must lie in one of the intervals

$$I_t = \left[ \frac{2B + tn}{s},\ \frac{3B - 1 + tn}{s} \right]$$

for some integer $t$. The attacker does not know $t$ but can compute all possible $t$ for which $I_t$ intersects the current $M$. The intersection over all valid $s$ eventually forces the interval to a unique integer.

### Complexity and Impact

Bleichenbacher showed that the attack typically requires about $10^5$ to $10^6$ oracle queries for a 1024-bit RSA key. This was feasible even in the late 1990s. The attack was successfully

applied to early SSL implementations, where a web server would reveal padding validity through different error messages or timing.

Nearly two decades later, the vulnerability resurfaced in a new variant called the **ROBOT attack** (Return Of Bleichenbacher's Oracle Threat), discovered in 2018. The ROBOT attack demonstrated that many modern TLS implementations still contained Bleichenbacher-style padding oracles; researchers found that almost a third of the top 100 domains were affected. The attack returned because TLS servers inadvertently leaked padding validity through different error codes or timing, and because the complexity of the TLS standard often led to incomplete or flawed implementations of the recommended countermeasures. This revival highlights that even well-known, decades-old vulnerabilities can remain exploitable if countermeasures are not implemented correctly.

#### Countermeasures

As discussed earlier in the padding section the PKCS #1 v1.5 padding is not provably secure. The RSA standard was updated to recommend OAEP (Optimal Asymmetric Encryption Padding), which uses a Feistel network with a hash function to randomise the message. OAEP is provably secure against chosen ciphertext attacks under the RSA assumption.

If PKCS #1 v1.5 must be used, the implementation must not leak any difference between valid and invalid padding. Specifically:

- Return the same error message for all decryption failures.
- Ensure the decryption time does not depend on padding validity (constant-time code).
- Avoid side channels such as cache behaviour or power consumption.

#### Conclusion

Bleichenbacher's attack is a paradigmatic example of how a seemingly harmless oracle—revealing only whether a ciphertext decrypts to a correctly padded message—can completely break RSA encryption. It forced a rethinking of padding schemes and influenced the design of modern cryptographic protocols that now use provably secure constructions like OAEP.

## 5.6 Lessons from a Quarter-Century of RSA Attacks

The RSA cryptosystem has withstood decades of mathematical cryptanalysis; the core problem of factoring large integers remains intractable. Yet the attacks surveyed in this chapter

demonstrate a sobering truth: **mathematical security is necessary but far from sufficient**. Real-world implementations introduce a vast attack surface that can completely subvert the algorithm's algebraic strength.

These attacks teach several enduring lessons:

1. **Implementations must be defensive, not optimistic.** Every computed result should be verified (e.g., by checking $s^e \equiv m \pmod{n}$ before output). Redundant computation, error-detecting codes, and blinding are not optional extras—they are essential.

2. **Side channels and oracles are the enemy.** Even a single bit of information—valid/invalid padding, timing difference, power consumption—can be amplified into a full break. Constant-time, uniform-response code is mandatory.

3. **Hardware matters.** Fault injection is a practical threat for any device an attacker can physically access. Countermeasures must span the entire stack, from the silicon (voltage/clock sensors) up to the application.

4. **Old attacks never die; they only return with new names.** Bleichenbacher's attack lay dormant for 20 years before ROBOT proved that developers still fail to implement the simple countermeasures. The same is likely true for fault attacks: as IoT and embedded devices proliferate, voltage-glitching will become a standard tool for extracting keys.

So we see that the gap between "textbook RSA" and "real RSA" is wide and treacherous. Every engineer who deploys RSA must internalise these attacks; otherwise, the mathematics that protects their data is just a facade.

# Chapter 6

# Conclusion

The long-standing security foundation of RSA – the assumed hardness of factoring – faces a fundamental and qualitative challenge from quantum computing. In 1994, Peter Shor discovered a quantum algorithm that solves the Integer Factorization Problem in polynomial time, specifically $O((\log n)^3)$ operations. Unlike classical algorithms, Shor's algorithm does not rely on sub-exponential heuristics; it exploits quantum parallelism and the quantum Fourier transform to find the period of a modular exponential function. This period directly reveals the factors of $n$. If a sufficiently large, fault-tolerant quantum computer were built, Shor's algorithm would break RSA completely, regardless of key size. The security margin provided by increasing the bit-length of $n$ becomes irrelevant against a quantum adversary, because the running time of Shor's algorithm grows only polynomially with the number of bits. While current quantum computers are far too small and error-prone to factor cryptographically relevant numbers, progress in quantum hardware and error correction is accelerating.

This quantum threat introduces the serious concern of "store now, decrypt later": encrypted data captured today can be stored with the expectation that future quantum computers will decrypt it. For information that must remain confidential for decades, this is already a pressing risk. Nevertheless, until large-scale quantum computers become a reality, RSA remains a cornerstone of modern cybersecurity when implemented correctly — using sufficiently large primes, proper padding (e.g., OAEP), and secure key management. In its properly implemented form, RSA is extraordinarily safe for protecting sensitive data today.

We hope this work has been useful and beneficial, and we have endeavoured to cover the subject of RSA cryptosystems and cryptanalysis from all essential angles. As with any technical survey, errors or oversights may remain; if you spot any mistake or wish to suggest

improvements, please do not hesitate to inform us. Your feedback is greatly appreciated.

# Bibliography


[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[2] G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*, 6th ed., Oxford University Press, 2008.

[3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.

[4] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed., CRC Press, 2020.

[5] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer, 2010.

[6] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1998.

[7] J. Hoffstein, J. Pipher, and J. H. Silverman, *An Introduction to Mathematical Cryptography* (Undergraduate Texts in Mathematics), Springer, 2008.

[8] D. Boneh, "Twenty years of attacks on the RSA cryptosystem," *Notices of the AMS*, vol. 46, no. 2, pp. 203–213, 1999.

[9] M. J. Wiener, "Cryptanalysis of short RSA secret exponents," *IEEE Transactions on Information Theory*, vol. 36, no. 3, pp. 553–558, 1990.

[10] D. Boneh, G. Durfee, and Y. Frankel, "An attack on RSA given a small fraction of the private key bits," in *Advances in Cryptology – ASIACRYPT '98*, Springer, 1998, pp. 25–34.

[11] D. Bleichenbacher, "Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1," in *Advances in Cryptology – CRYPTO '98*, Springer, 1998, pp. 1–12.

[12] D. Brumley and D. Boneh, "Remote timing attacks are practical," *Computer Networks*, vol. 48, no. 5, pp. 701–716, 2005.

[13] J. M. Pollard, "A Monte Carlo method for factorization," *BIT Numerical Mathematics*, vol. 15, no. 3, pp. 331–334, 1975.

[14] I. K. Salah, A. Darwish, and S. Oqeili, "Mathematical attacks on RSA cryptosystem," *Journal of Theoretical and Applied Information Technology*, vol. 96, no. 15, 2018.

[15] A. Nitaj, "The mathematical cryptography of the RSA cryptosystem," in *Algebra, Codes and Cryptology*, Springer, 2019.

[16] A. Nitaj, M. R. K. Ariffin, D. I. Nassr, and H. M. Bahig, "New attacks on the RSA cryptosystem," in *Progress in Cryptology – AFRICACRYPT 2014*, Springer, 2014.

[17] L. Euler, "Theoremata arithmetica nova methodo demonstrata," *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, vol. 8, pp. 74–104, 1736.

[18] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of eliminating errors in cryptographic computations," *Journal of Cryptology*, vol. 14, no. 2, pp. 101–119, 2001.

[19] D. Perrin, *Cours d'algèbre*, Ellipses, 1996.

[20] H. Hanche-Olsen, "Extended Euclid algorithm," lecture notes, 2010.

[21] N. T. Lionel, "Mémoire de Master II (Algèbre)," Université de Yaoundé I, 2017–2018.

[22] B. Kleinberg, "The Miller–Rabin randomized primality test," Lecture notes, CS 482, Cornell University, 5 May 2010.

[23] M. Bläser and C. Saha, "Computational number theory and algebra," Lecture 18, 20 June 2012.

[24] A.-S. Charest, "Pollard's $p-1$ and Lenstra's factoring algorithms," Technical report, 2005.

[25] J. Pité and Y. Zhong, "The RSA cryptosystem," Student project, mentor H. Zhu.

[26] K. Moriarty (Ed.), B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," IETF, RFC 8017, Nov. 2016. https://doi.org/10.17487/RFC8017

[27] H. Böck, J. Somorovsky, and C. Young, "ROBOT: Return of Bleichenbacher's Oracle Threat," Dec. 2017. https://robotattack.org/

[28] S. Nakov, "Secure Random Generators (CSPRNG)," in *Practical Cryptography for Developers*, https://cryptobook.nakov.com/secure-random-generators/secure-random-generators-csprng